



第9章

组合逻辑电路设计

Design Combinational Circuits

2020年11月17日

组合逻辑电路设计

一. 简介

1. 组合逻辑电路分析
2. 组合逻辑设计步骤

二. 比较器

三. 译码器

四. 编码器

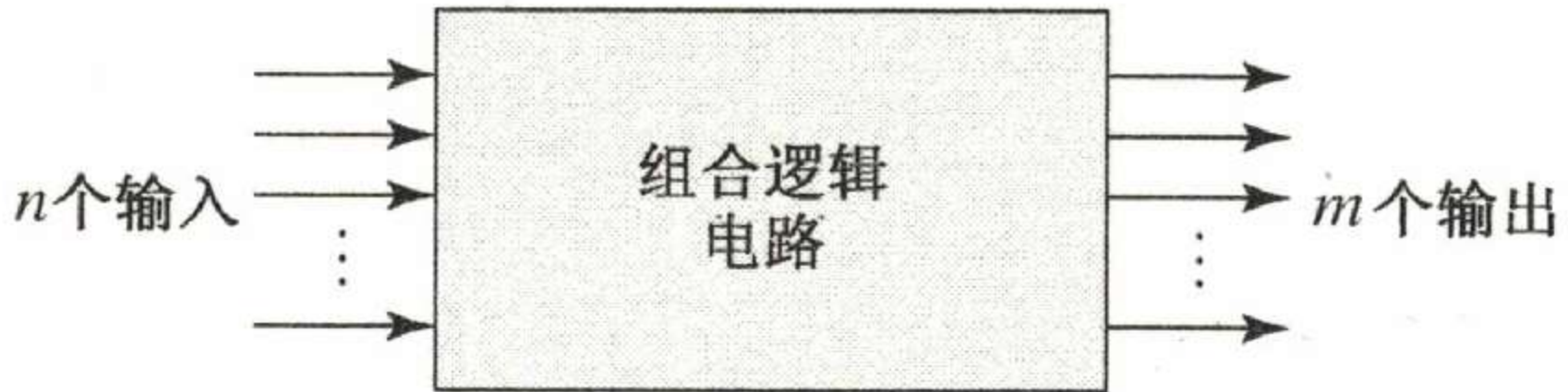
五. 多路选择器

六. 加法器

二进制加法器、BCD加法器、减法器

组合逻辑电路

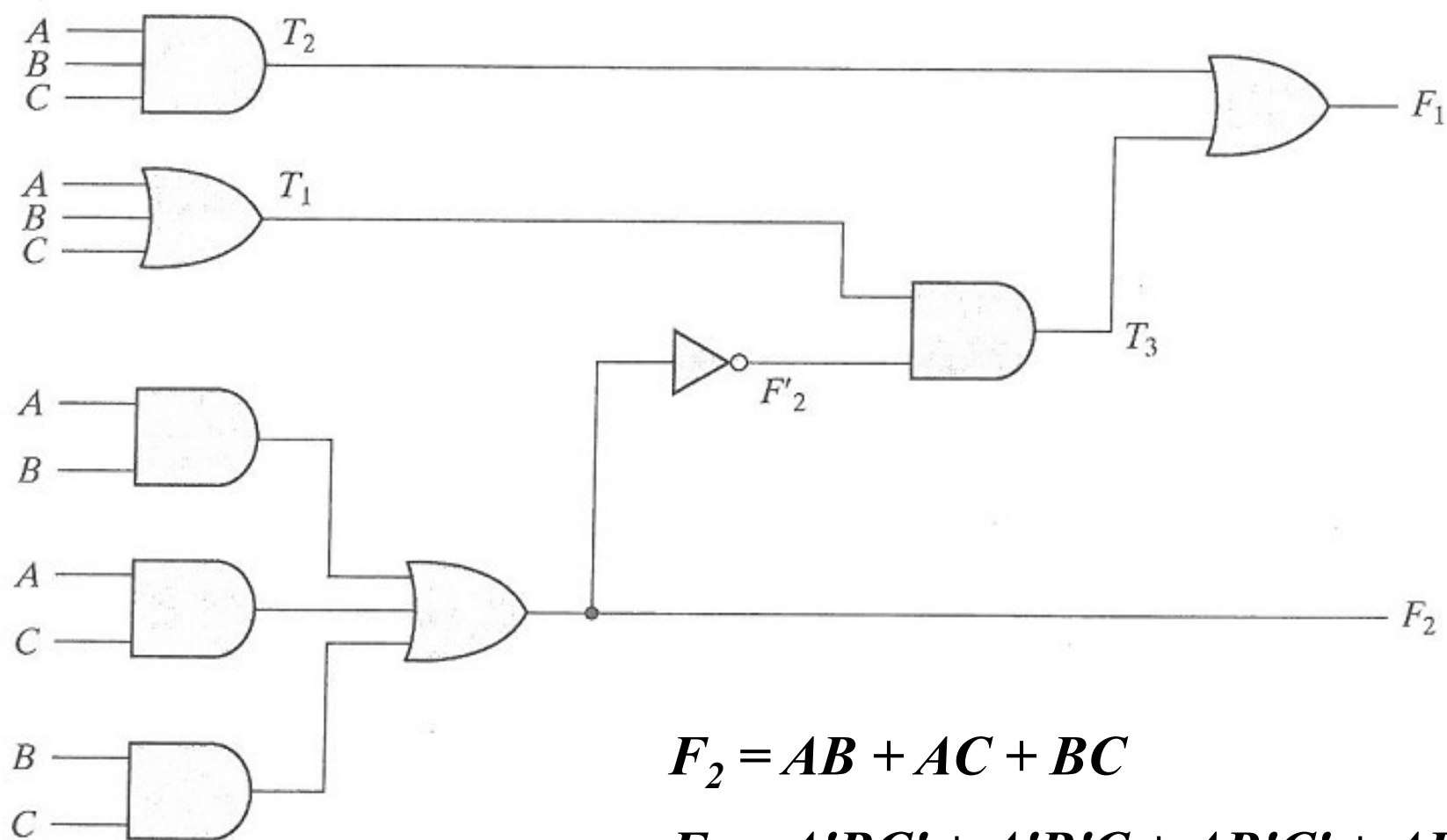
- 组合电路由相互连接的逻辑门构成，逻辑门接收输入信号后产生输出信号，将输入的二进制数据信息转换成需要的输出数据
- 组合电路可以用真值表来详细描述，也可由 m 个布尔函数来描述，一个函数对应一个输出变量，每个输出是 n 个输入变量的函数



组合逻辑电路分析

- 组合逻辑电路的分析就是确定组合电路的实现功能。通过分析给定的逻辑图，得到一组布尔函数、真值表或电路功能的说明
- 如果待分析的逻辑图具有函数名或者功能实现的说明，分析问题就简化为对所述功能的验证
- 从逻辑图得到输出布尔函数的基本步骤如下：
 1. 将所有输入变量函数的逻辑门输出用任意符号标出，确定每个门输出的布尔函数
 2. 标出输入变量函数的逻辑门，前面标注过的逻辑门采用其他任意符号，求出这些逻辑门的布尔函数
 3. 重复第 2 步的操作，直到求得电路的输出
 4. 把前面的函数重复代入，得到仅有输入变量的输出布尔函数

组合逻辑电路分析



组合逻辑电路分析

■ 直接从逻辑电路图得到真值表，步骤如下：

1. 确定电路中输入变量的个数 n ，建立 2^n 种可能的输入组合，并把这些从0到 2^n-1 之间的二进制数列在一张表上
2. 把选中门的输出用任意符号标示出来
3. 得到那些仅有输入变量函数的门的输出真值表
4. 继续求取先前定义过函数值的门的输出真值表，直到所有输出都确定为止

A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

组合逻辑电路设计

- 根据设计目标要求，画出逻辑电路图或得到逻辑图的一组布尔函数。其步骤如下：
 1. 根据电路要求，确定需要的输入数和输出数，并给每个变量分配一个符号
 2. 列出输入和输出之间关系的真值表
 3. 求输出函数的最简表达式
 4. 画出逻辑电路图，并验证设计的正确性
- 真值表中定义的输出函数对组合电路做了准确描述，将设计要求正确翻译成真值表是很重要的一步
- 对真值表所列出的输出二进制函数要设法进行化简，有多种不同的简化表达式可供选择

组合逻辑电路设计

- 设计举例：码变换，把二-十进制（**BCD**）码转化为余 **3** 码

二进制码

- 数字系统不仅能表示和处理二进制数，而且还能够表示和处理其他的离散信息元素
- 在一组数值中，任何不同的离散信息元素都可以用二进制码（一种**0**和**1**的组合模式）来表示
- 一个 **n** 位二进制码就是一个 **n** 位的比特组合，它有 2^n 种不同的 **0**、**1**组合，每种组合都代表一个编码
- 给 2^n 个不同的值编码需要的最小位数是 **n** ：一个四元素集可以用两个比特来编码；八元素集需要三个比特；而十六元素集则需要四个比特； ...

BCD码

- 区分**10**个元素的二进制码至少要有**4**位，而**4**位二进制码可以有**16**种不同的组合。因此，其中将有**6**种编码没有被分配。将**4**位排列为**10**种不同的组合就能得到不同的二进制码
- 十进制数最常用的码对应的二进制数分配，是二进制编码十进制数(二-十进制码)，简称为**BCD码**

十进制符	BCD码
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

其它的十进制码

十进制符	BCD码	2421码	余3码	8,4,-2,-1码
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111

其它的十进制码

- 有权码：每位都被赋予了一个权值，每个数可以通过将码组中所有为 **1** 的权值相加来求得
 - **BCD 码** 和 **2421 码** 是有权码
 - **8,4,-2,-1 码** 是一种具有正、负权值的十进制码
- 自补码：其十进制数反码是通过将该码中 **0** 和 **1** 变成 **1** 和 **0** 直接得到的，即它们的 **0** 和 **9**、**1** 和 **8**、**2** 和 **7**、**3** 和 **6**、**4** 和 **5** 等均互为反码
 - 例如：十进制数 **395** 用余 **3** 码表示为 **0110 1100 1000**，而 **604** 的反码表示为 **1001 0011 0111**

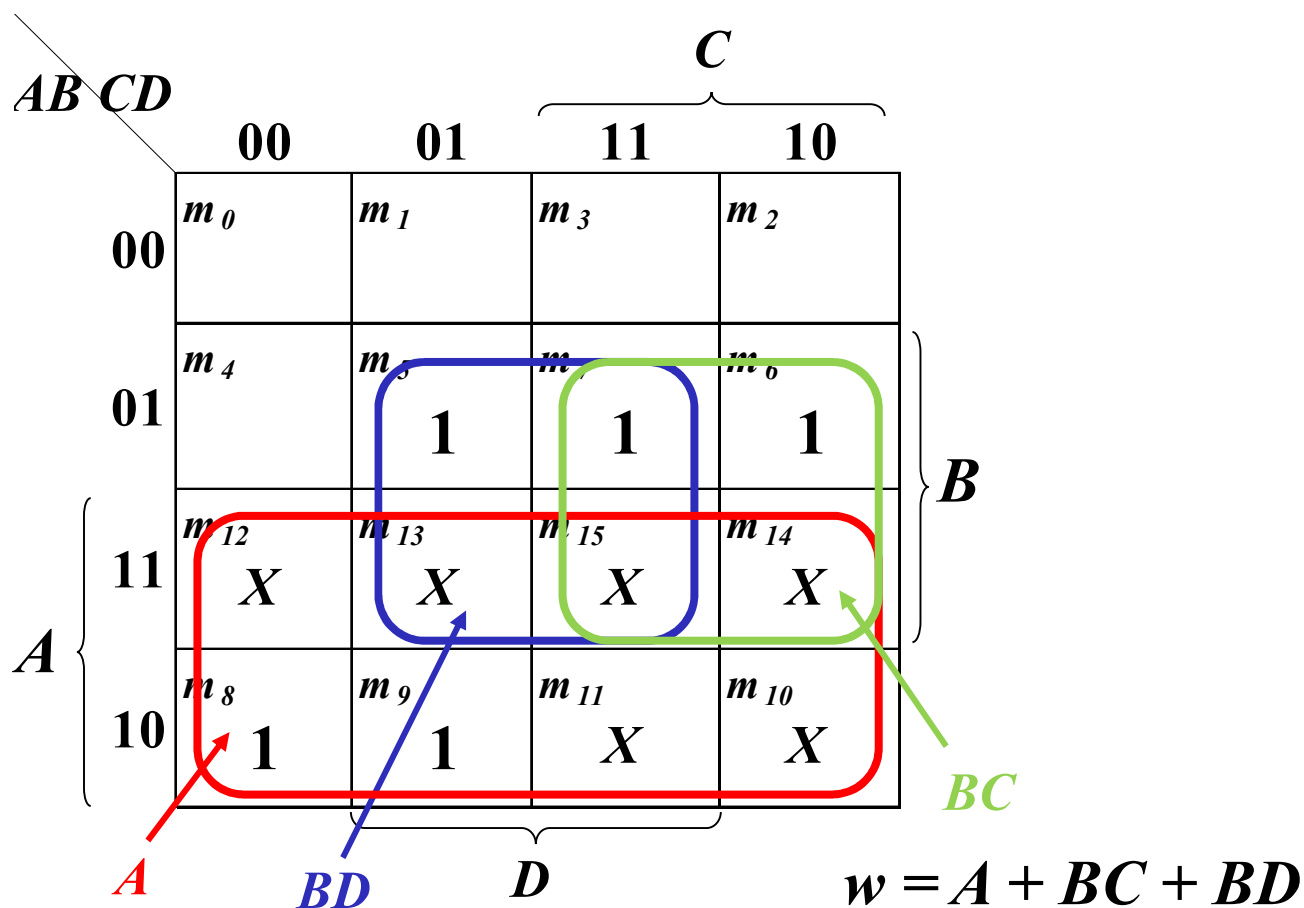
组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余 3 码

输入 BCD 码				输出余 3 码			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

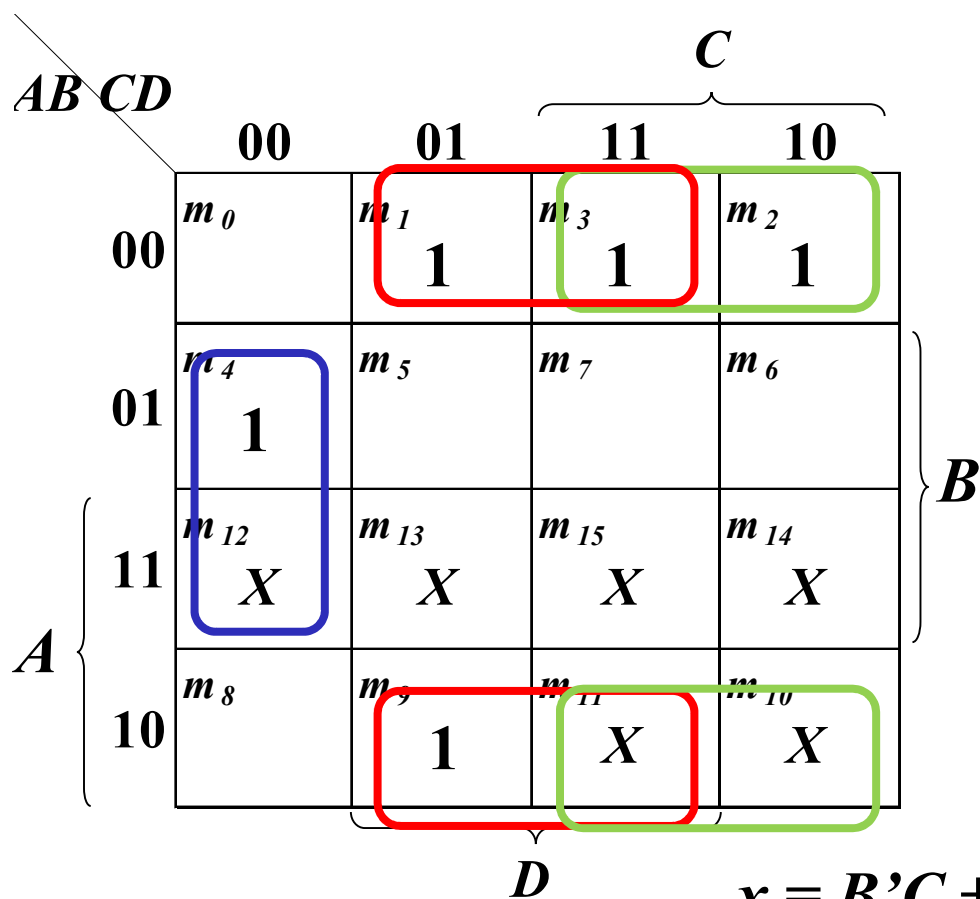
组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余 3 码



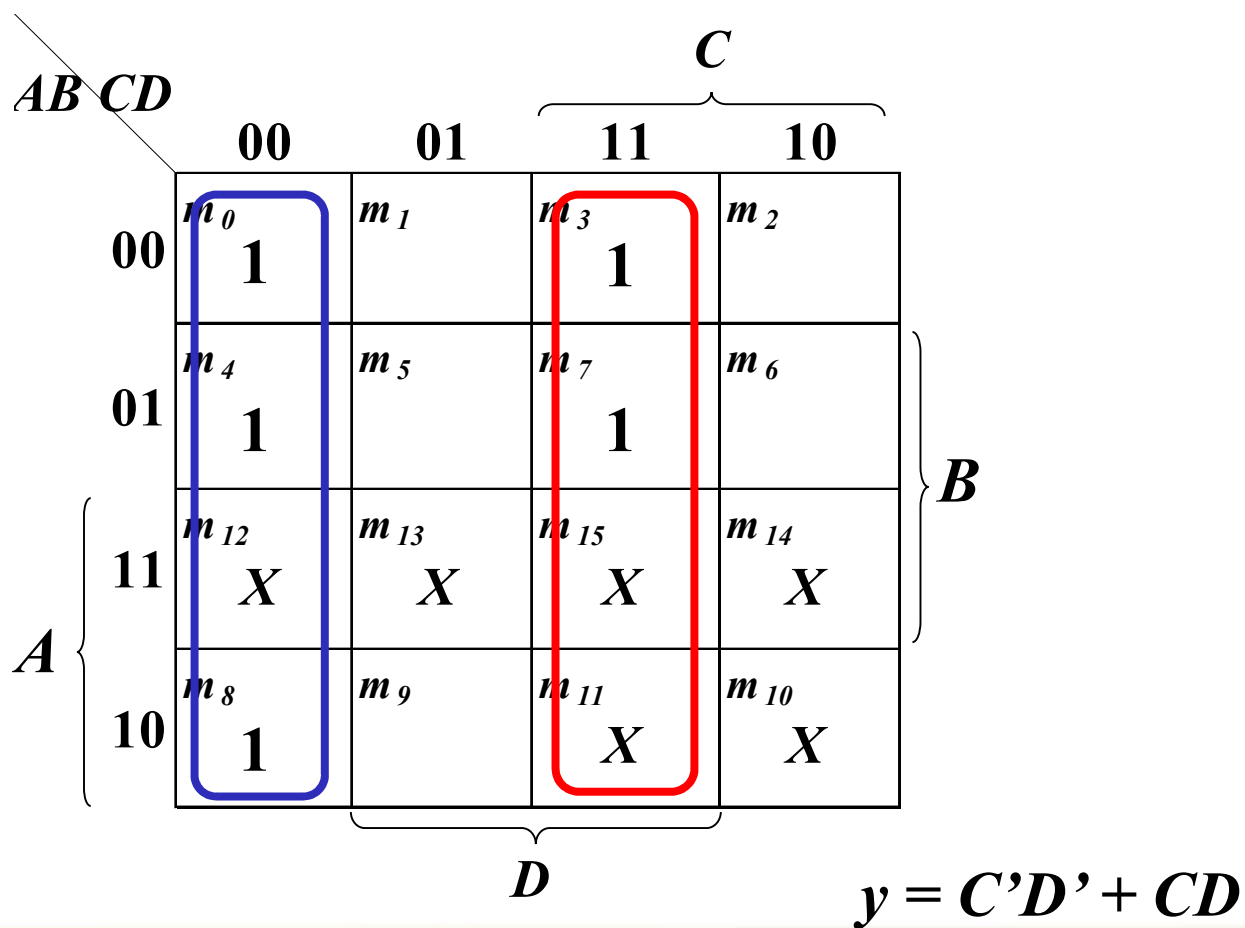
组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余 3 码



组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余 3 码



组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余 3 码

$AB \backslash CD$		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X
		D			

$z = D'$

组合逻辑电路设计

■ 举例：码变换，把二-十进制（BCD）码转化为余 3 码

- 从化简后的布尔表达式可以直接得到每个输出的两级门电路逻辑图，也可以对从卡诺图得到的表达式进行代数变换，用其他的逻辑图用来实现这个电路

$$z = D'$$

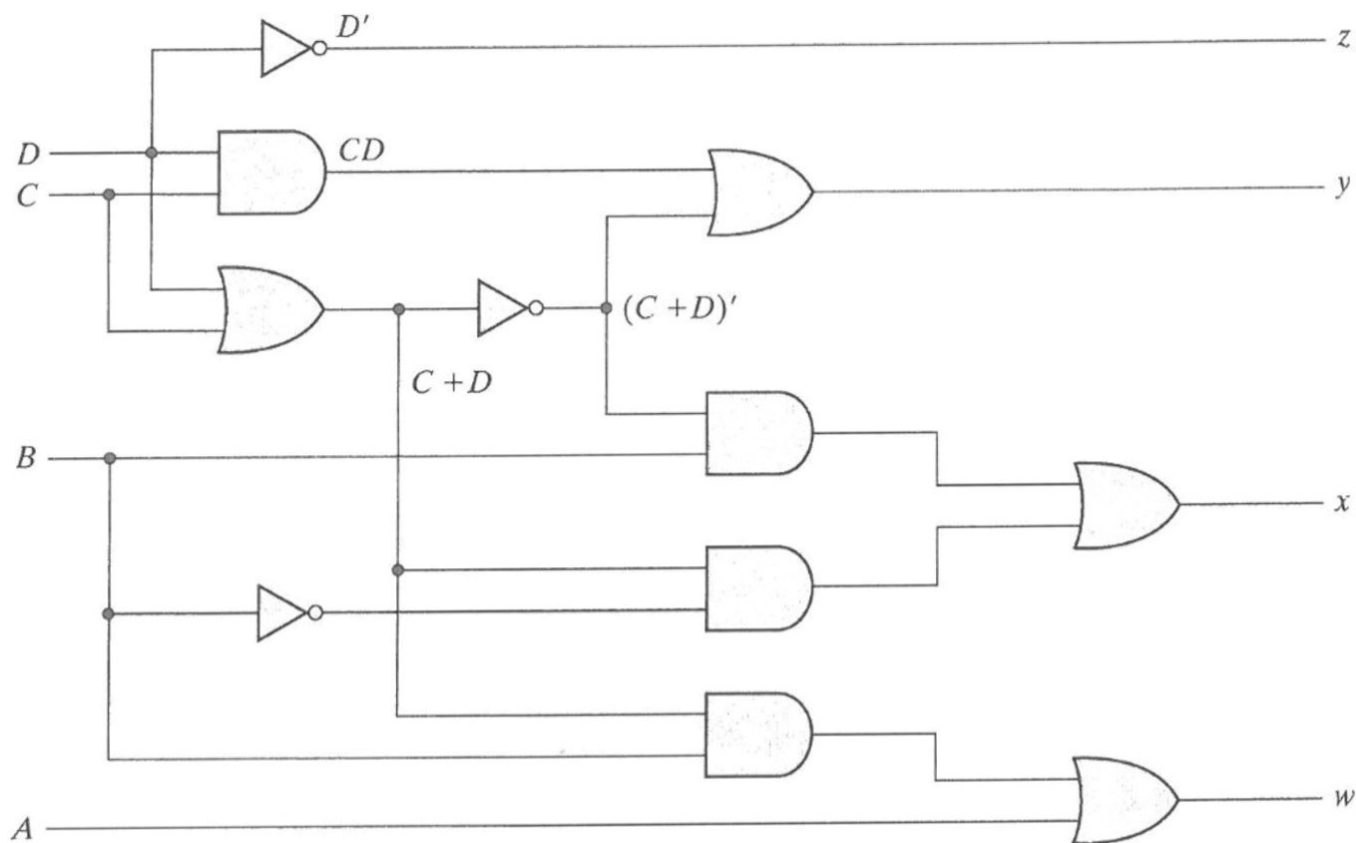
$$y = CD + C'D' = CD + (C + D)'$$

$$\begin{aligned} x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\ &= B'(C + D) + B(C + D)' \end{aligned}$$

$$w = A + BC + BD = A + B(C + D)$$

组合逻辑电路设计

- 举例：码变换，把二-十进制（BCD）码转化为余3码



组合逻辑电路设计

一. 简介

二. 比较器

三. 译码器

四. 编码器

五. 多路选择器

六. 加法器

数值比较器

- 两个数的比较就是确定一个数是大于、小于还是等于另一个数，数值比较器是对两个数 A 和 B 进行比较，以确定它们相对大小的组合电路，比较结果是由三个二进制变量区分 $A > B$ 、 $A = B$ 和 $A < B$ 三种情况
- 对于两个4位二进制数： $A = A_3A_2A_1A_0$ 和 $B = B_3B_2B_1B_0$
 - 如果所有对应的位都相等，则这两个数相等。每组的相等关系可以用异或函数表示
$$x_i = A_iB_i + A'_iB'_i, \quad i = 0, 1, 2, 3$$
$$(A = B) = x_3x_2x_1x_0$$
只有当两个数的所有对应位都相等时，该二进制变量 ($A=B$) 才等于 1

数值比较器

■ 对于两个4位二进制数： $A=A_3A_2A_1A_0$ 和 $B=B_3B_2B_1B_0$

- 为了确定 A 和 B 的大小，从最高位开始检查对应的每组数的相对大小。如果两个数相等，就比较较低一位的一组数，一直比较到数不相等为止

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

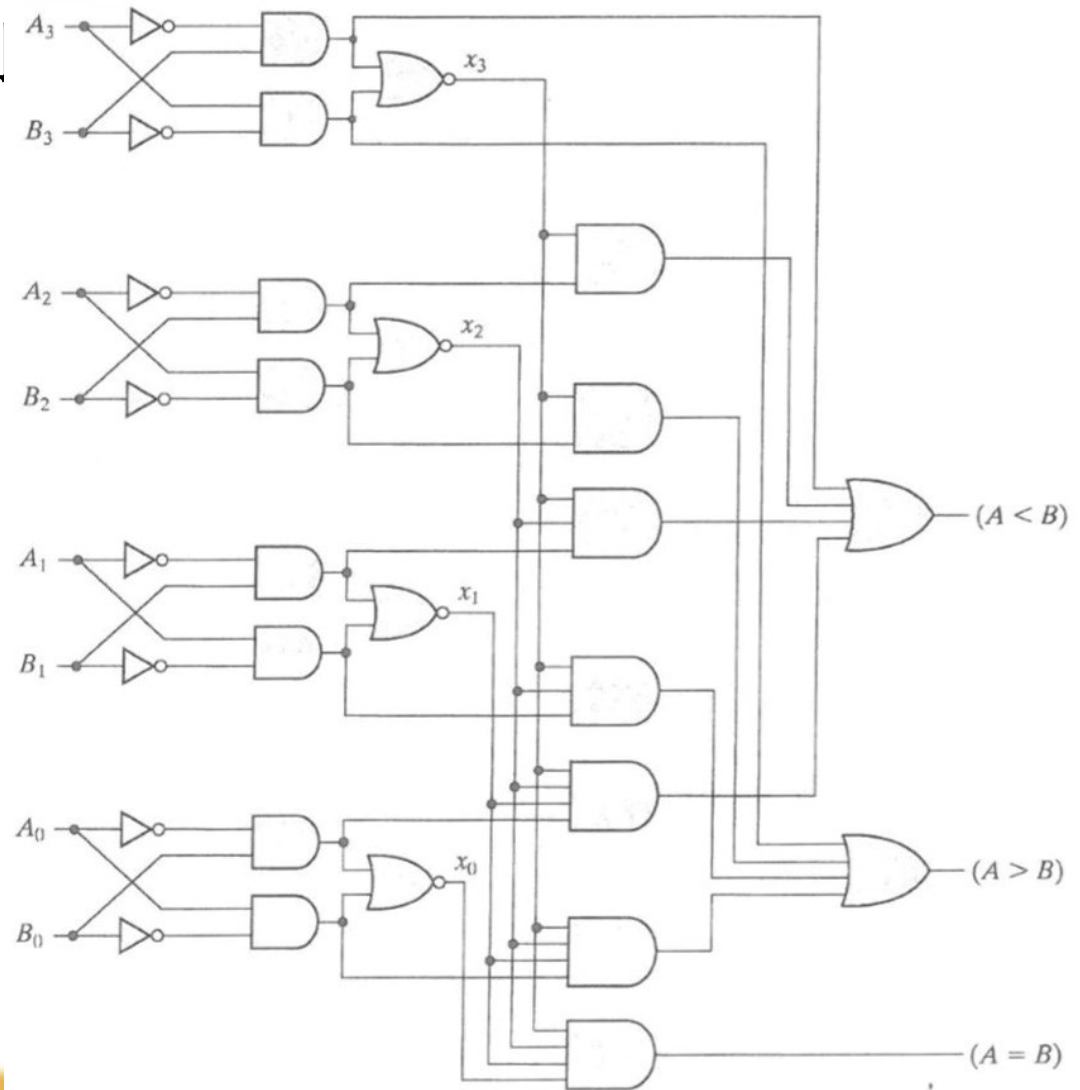
$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

符号 $(A > B)$ 和 $(A < B)$ 是二进制输出变量，分别在 $A > B$ 或 $A < B$ 时等于1

$(A = B)$ 、 $(A > B)$ 和 $(A < B)$ 三个输出变量逻辑门实现比较简单，包含了一定数量的重复元件，不相等的输出可以共用产生相等的输出的逻辑门

数值比较器

- 对于两个4位二进制

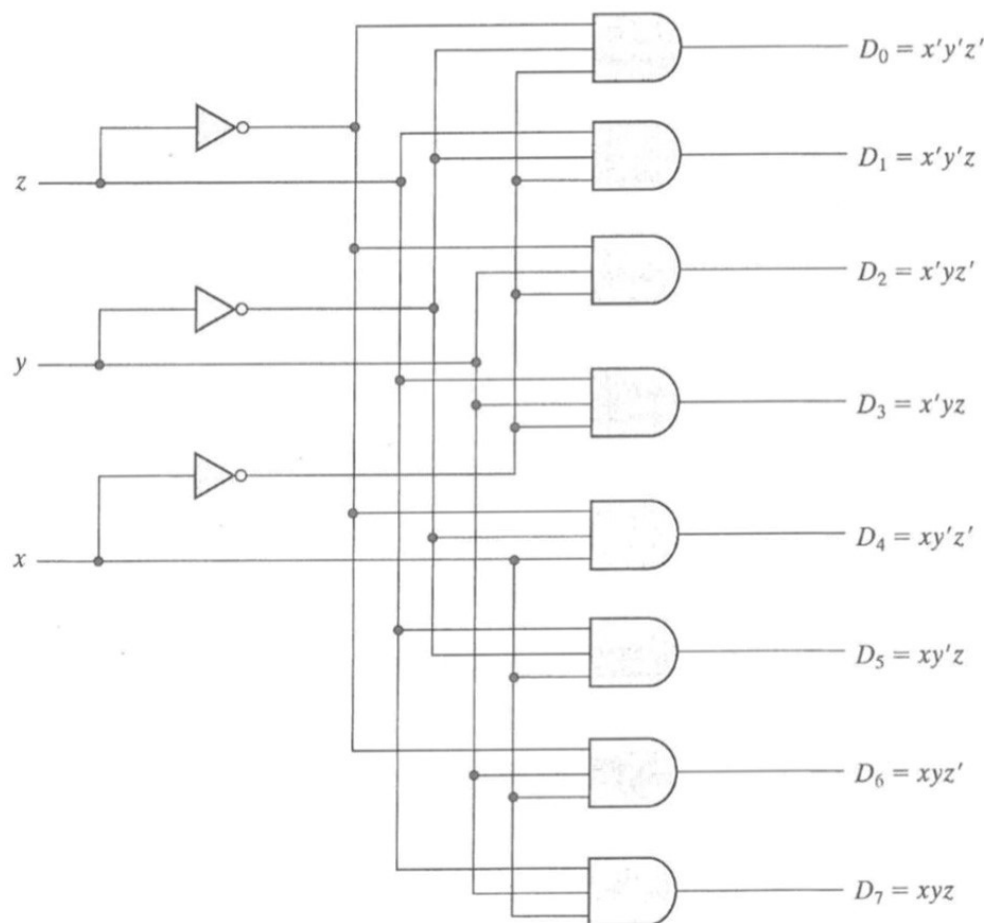


组合逻辑电路设计

- 一. 简介
- 二. 比较器
- 三. 译码器
- 四. 编码器
- 五. 多路选择器
- 六. 加法器

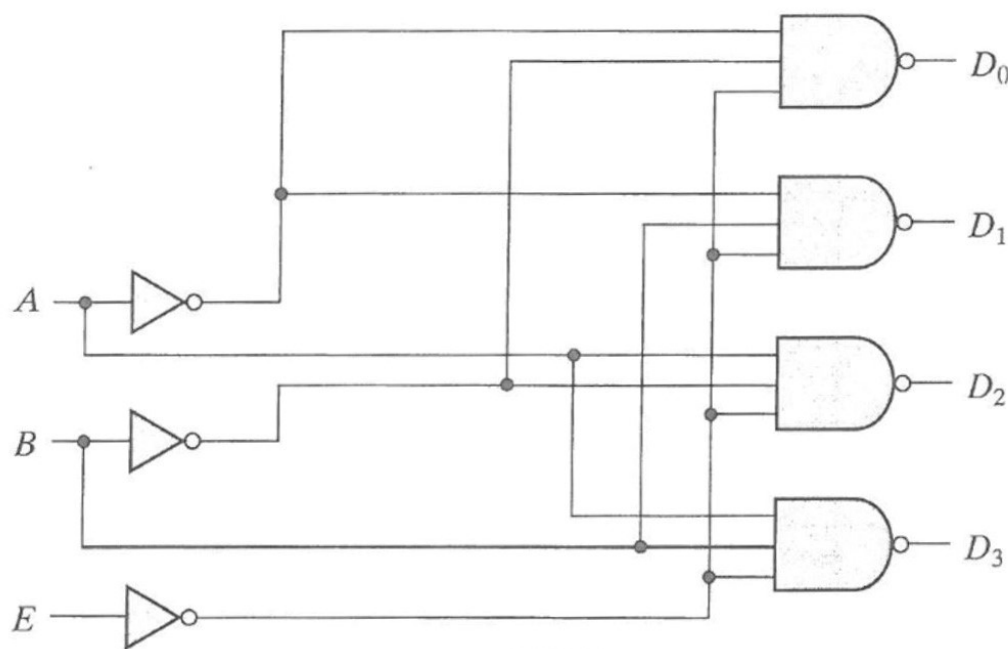
译码器

- 译码器就是一个把来自 n 条输入线的二进制信息转化为 2^n 条输出线的组合电路，称为 n - m 线译码器。
- 例：3-8线译码器



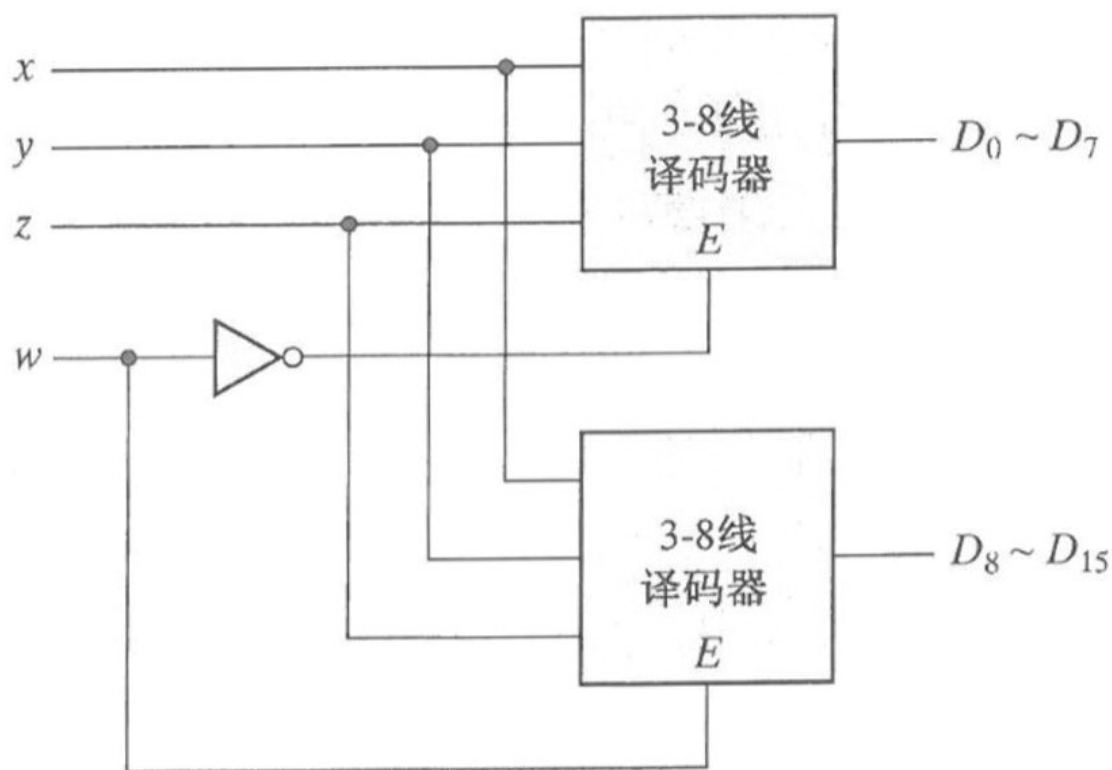
译码器

- 译码器可由与非门构成，以反相形式来产生译码器的最小项非常经济，也可以加上一个或多个使能端控制电路的工作
- 例：由与非门构成的带有一个使能输入的 **2-4** 线译码器



译码器

- 带使能端的译码器可以连在一起形成一个大型的译码器。例：两个带使能端的**3-8** 线译码器连在一起构成了一个**4-16** 线译码器



译码器

■ 用译码器实现组合逻辑

- 译码器提供 n 个输入变量的 2^n 个最小项，其输出由每一组输入唯一确定。任何布尔函数可以表示成最小项之和，就可以用译码器产生最小项，并且用外部或门来产生逻辑和
- 任何有 n 个输入、 m 个输出的组合电路就可以用一个 2^n 译码器和 m 个或门来实现
- 实现步骤：先把电路的布尔函数用最小项之和的形式表示出来，然后选择译码器来产生输入变量的所有最小项，每个或门的输入从函数最小项译码器的输出中选择
- 例：全加器电路的实现

采用最小项之和表示的组合电路的函数：

$$S(x, y, z) = \sum(1, 2, 4, 7),$$

$$C(x, y, z) = \sum(3, 5, 6, 7)$$

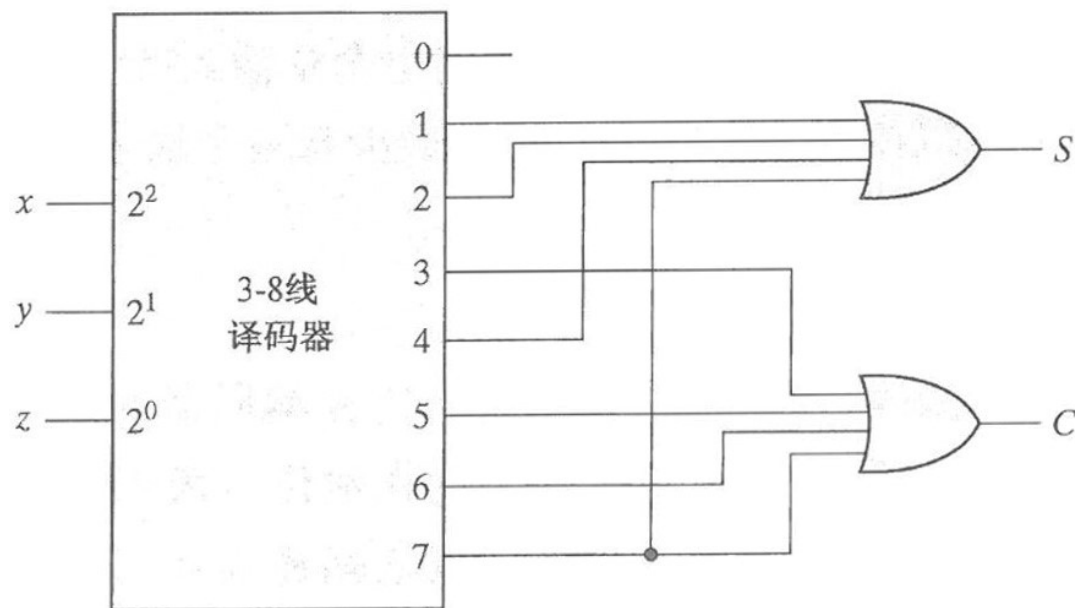
译码器

■ 用译码器实现组合逻辑

● 例：全加器电路的实现

采用最小项之和表示的组合电路的函数：

$$S(x, y, z) = \sum(1, 2, 4, 7), \quad C(x, y, z) = \sum(3, 5, 6, 7)$$



组合逻辑电路设计

- 一. 简介
- 二. 比较器
- 三. 译码器
- 四. 编码器
- 五. 多路选择器
- 六. 加法器

编码器

- 编码器是一种实现译码器反运算的数字电路，有 2^n 个输入线和 n 个输出线。例：8-3线编码器

输 入								输 出		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

- 局限：

1) 任何时候只有一个输入是有效的； 2) 全零输入问题

编码器

■ 优先编码器

- 包含优先级函数的编码器电路。如果有两个或多个输入同时有效，拥有最高优先级的输入将处于优先地位
- 例：4输入优先级编码器

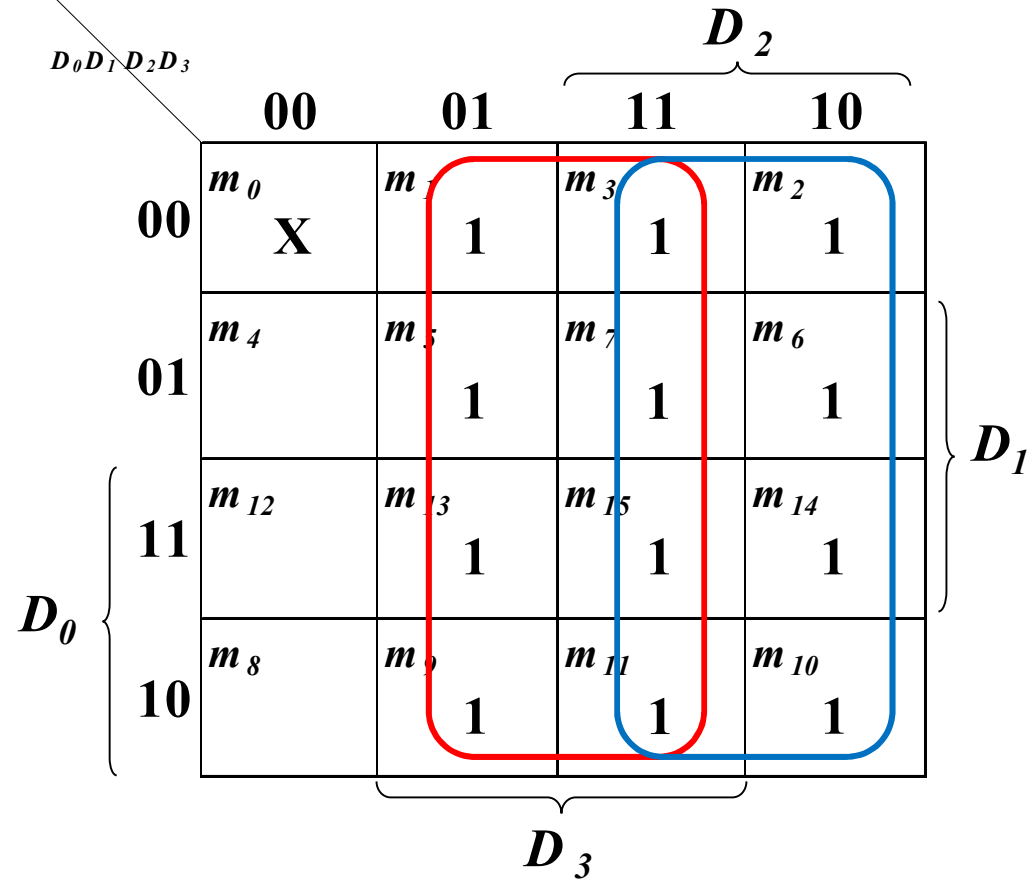
输 入				输 出		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

下标数字越大，输入的优先级就越高。输入 D_3 具有最高的优先级
V：有效位指示符

编码器

■ 优先编码器

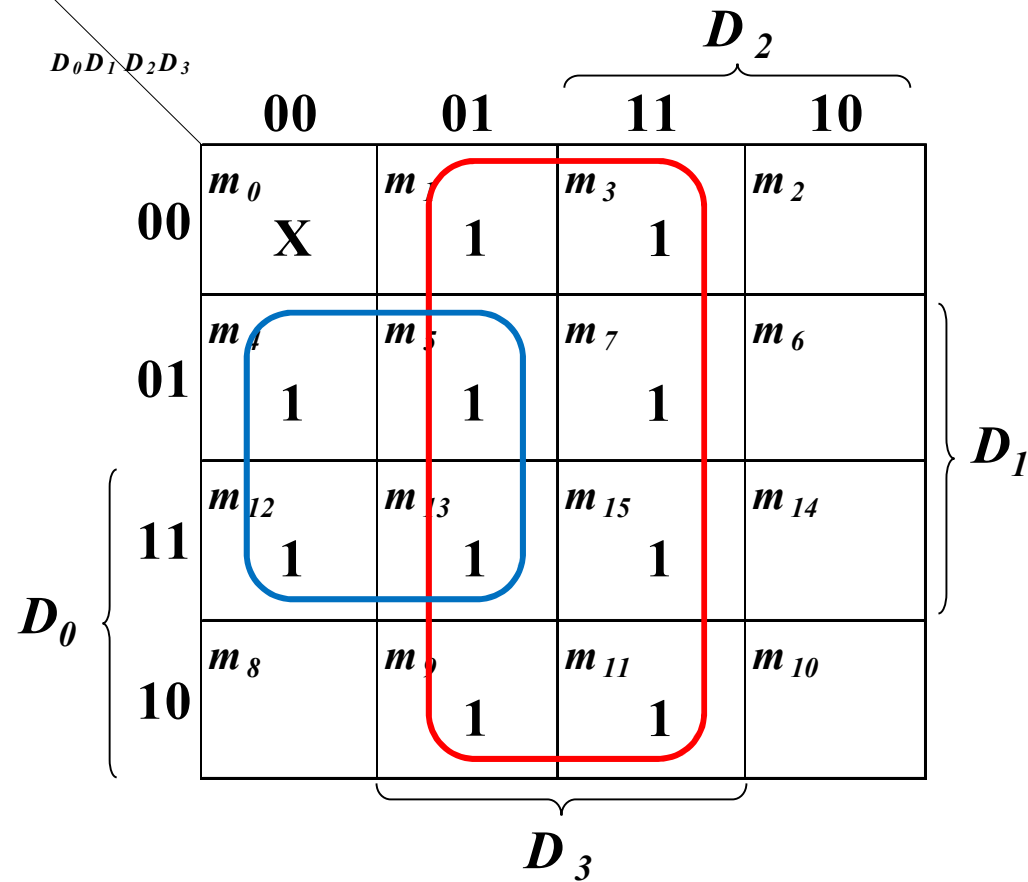
● x和y的化简



编码器

■ 优先编码器

● x和y的化简



$$y = D_1 D_2' + D_3$$

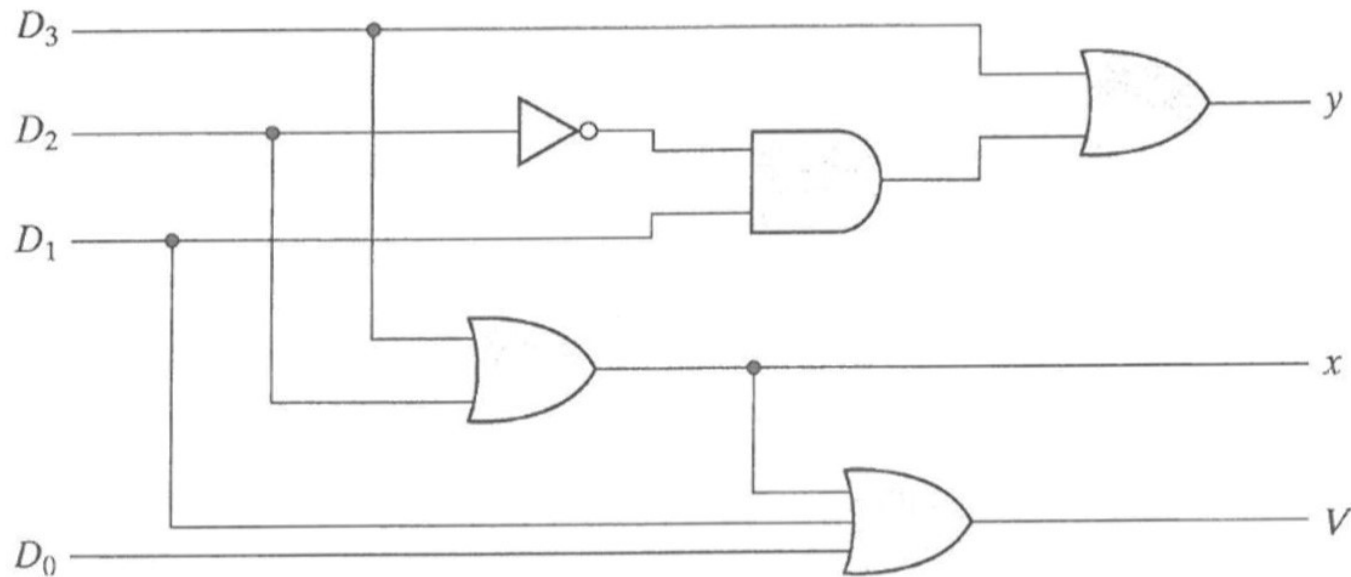
编码器

■ 优先编码器

$$x = D_2 + D_3$$

$$y = D_1 D_2' + D_3$$

$$V = D_0 + D_1 + D_2 + D_3$$

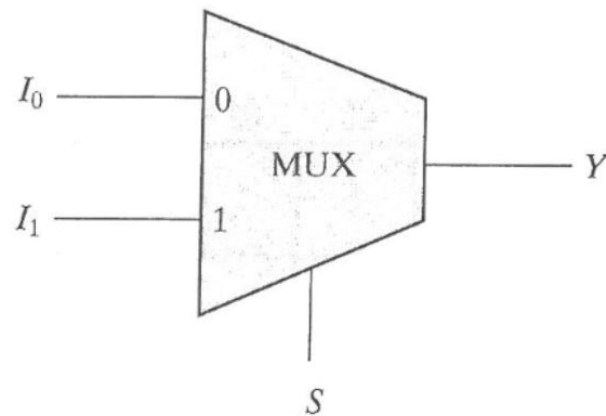
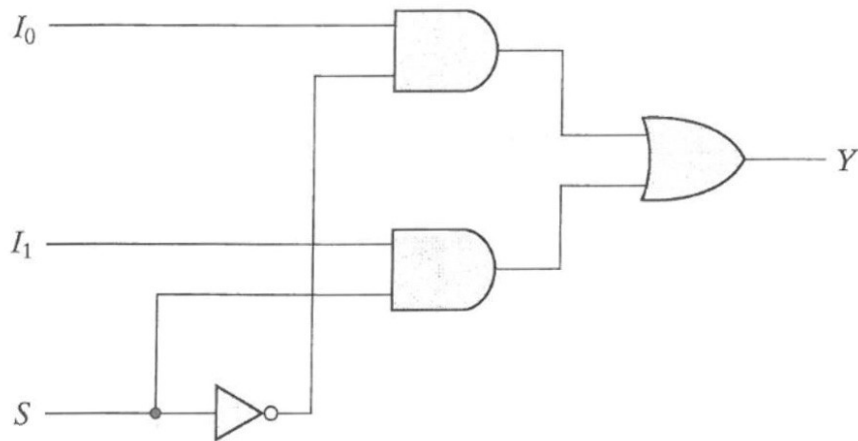


组合逻辑电路设计

- 一. 简介
- 二. 比较器
- 三. 译码器
- 四. 编码器
- 五. 多路选择器
- 六. 加法器

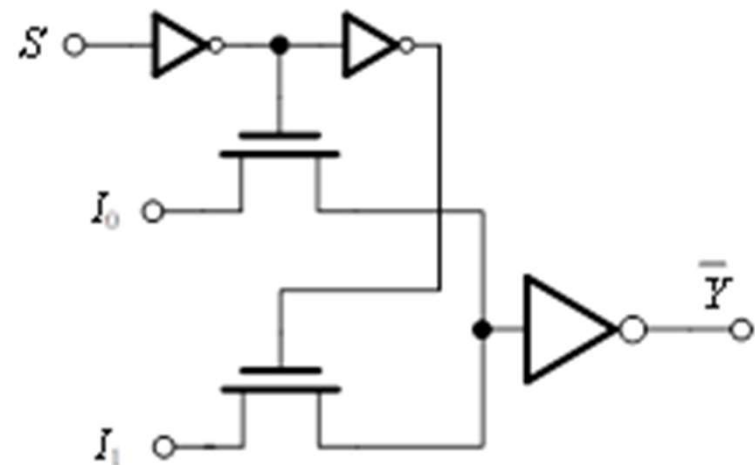
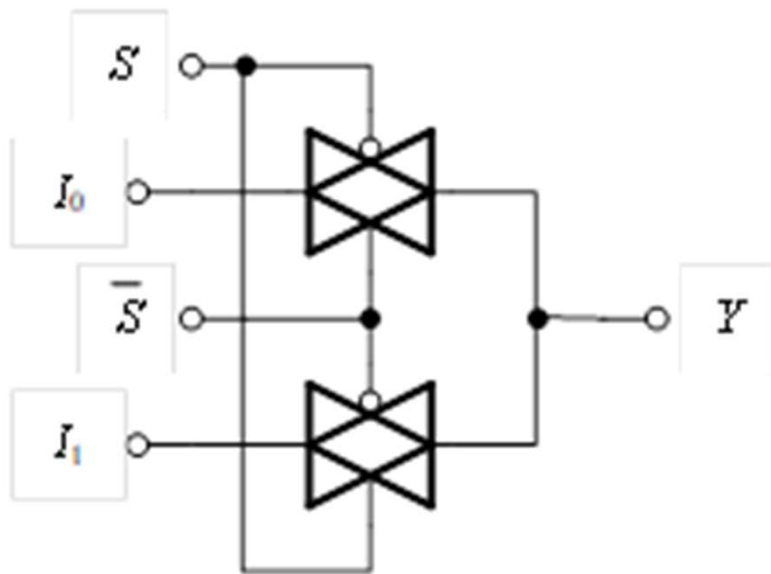
数据选择器

- 数据选择器是从多路输入线中选择其中的一路二进制信息到输出线上的一种组合电路。由一组选择线来控制，通常有 2^n 条输入线和 n 条选择线，选择线的组合决定哪条输入线被选中
- 二选一数据选择器



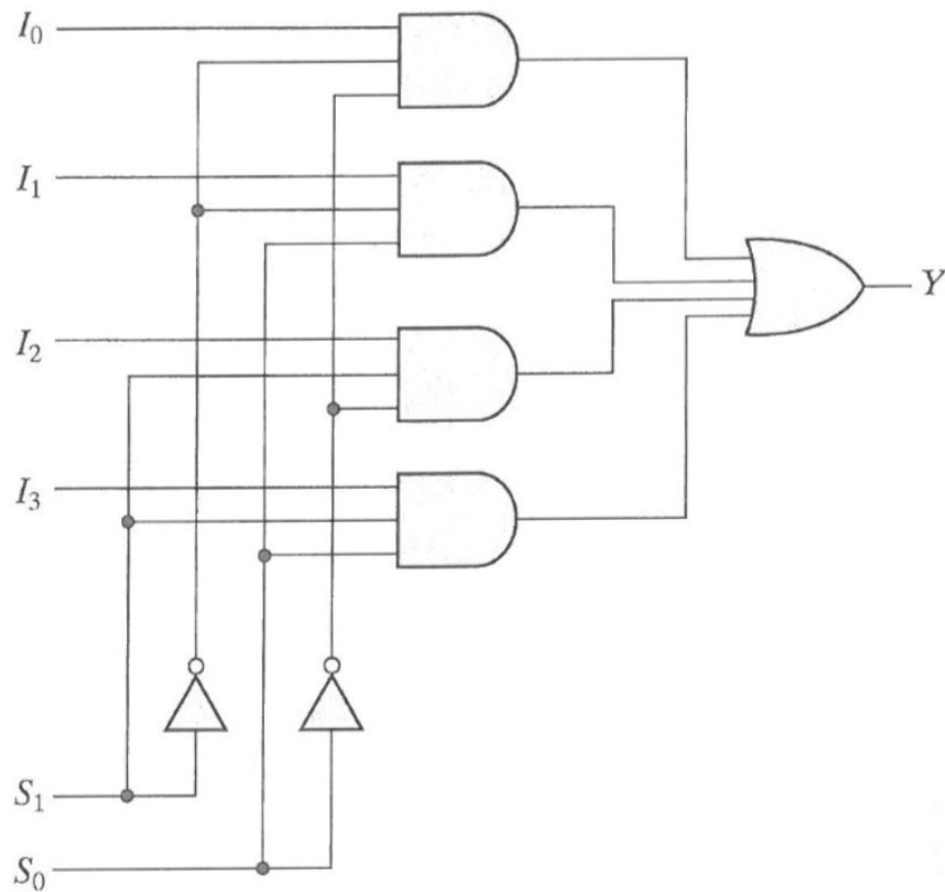
数据选择器

■ 传输门和传输管实现二选一数据选择器



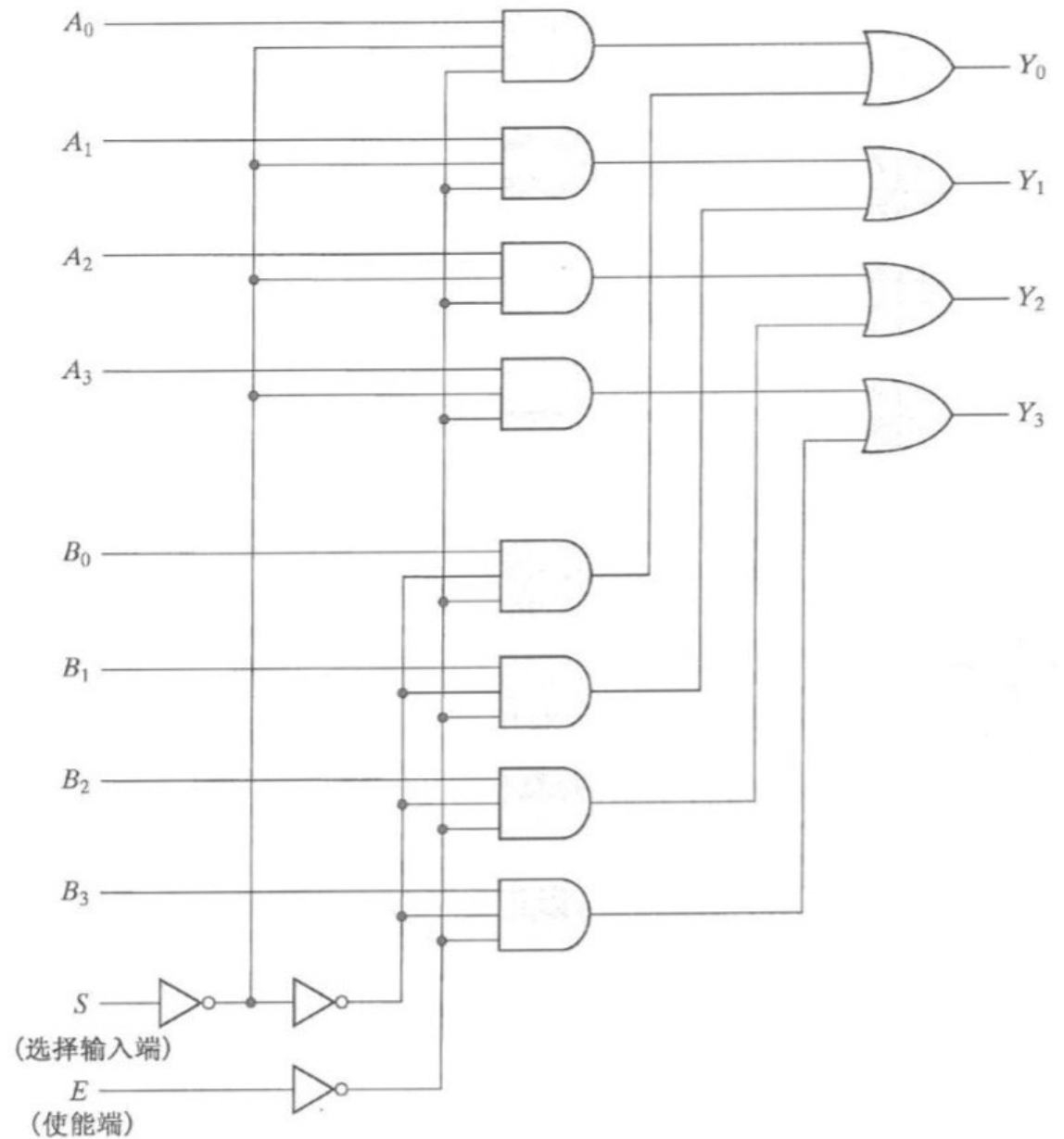
数据选择器

■ 四选一数据选择器



数据选择器

■ 4位二选一 数据选择器



数据选择器

■ 用数据选择器实现布尔函数

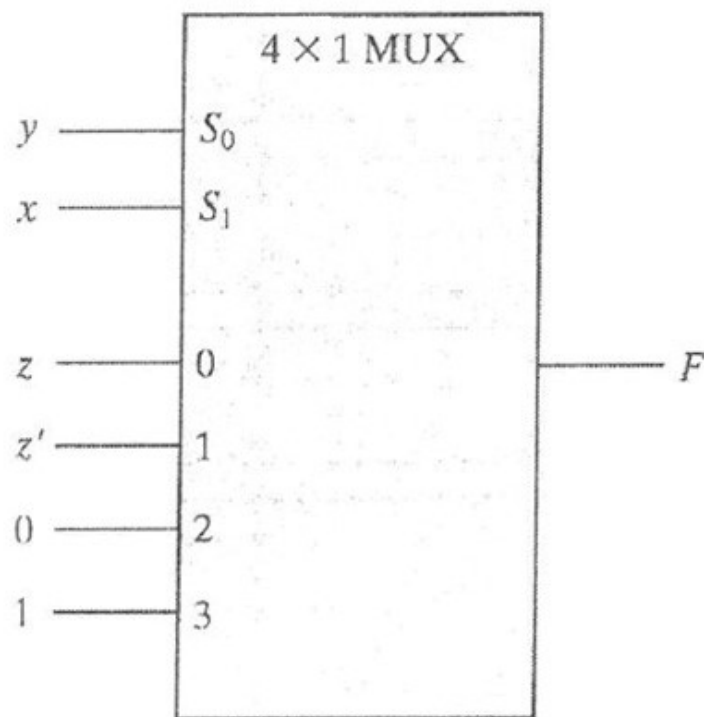
- 数据选择器实质上就是一个带或门的译码器。函数的最小项可由与选择输入相连的电路产生，单个最小项可以由输入数据来选择
- 可以用一个有 n 个选择输入和 2^n 个数据输入的数据选择器来实现布尔函数，其中每个数据输入对应一个最小项
- 另一种更有效的方法：用一个有 $n-1$ 个选择输入的数据选择器来实现 n 个变量的布尔函数，函数的前 $n-1$ 个变量连到数据选择器的选择输入端，剩下的一个变量用作数据输入。
- 例： $F(x, y, z) = \sum(1, 2, 6, 7)$

数据选择器

■ 用数据选择器实现布尔函数

● 例: $F(x, y, z) = \Sigma(1, 2, 6, 7)$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	



数据选择器

■ 用数据选择器实现布尔函数

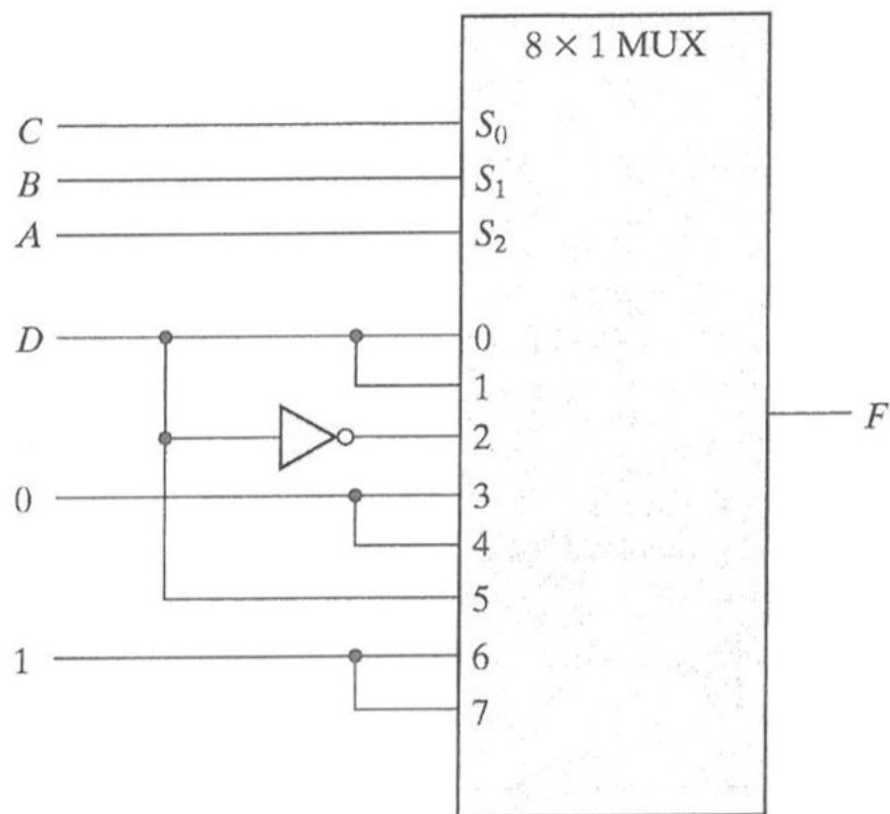
- 用 $n-1$ 个选择输入和 2^{n-1} 个数据输入的数据选择器实现任意 n 个变量的布尔函数一般步骤：
 - ◆ 首先列出布尔函数的真值表，真值表的前 $n-1$ 个变量被用作数据选择器的选择输入
 - ◆ 对于输入变量的每个组合，把输出当作最后一个变量的函数，这个函数可以是0、1、原变量、反变量
 - ◆ 将这些值以正确的顺序加到数据输入端 D
- 例： $F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$

数据选择器

■ 用数据选择器实现布尔函数

- 例: $F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$

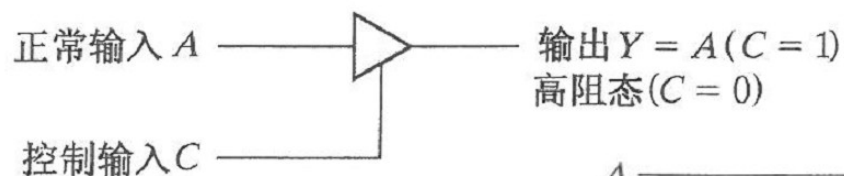
A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



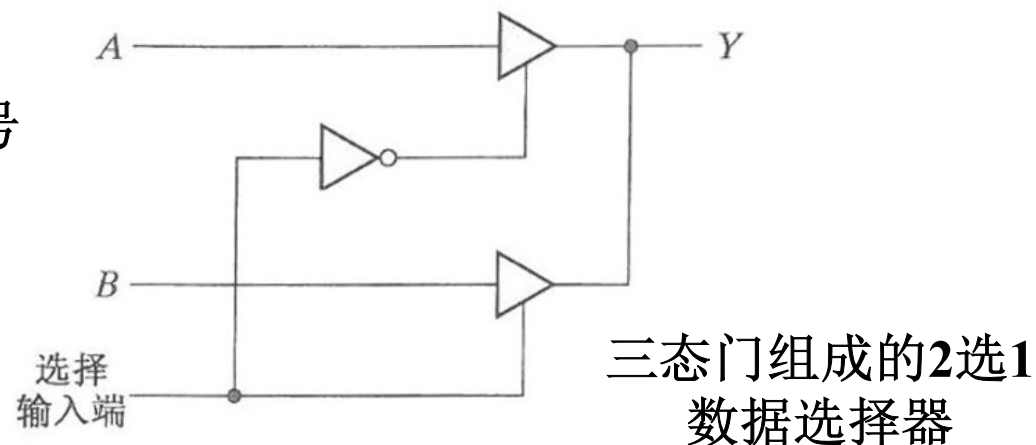
数据选择器

■ 三态门

- 数据选择器可以用三态门来构造
- 三态门是一种呈现三种状态的数字电路：逻辑1、逻辑0和高阻态
- 高阻态呈现为开路，也就是输出表现为无连接。由于这个特性，可以将大量的三态门输出用线连在一起形成一个共用线



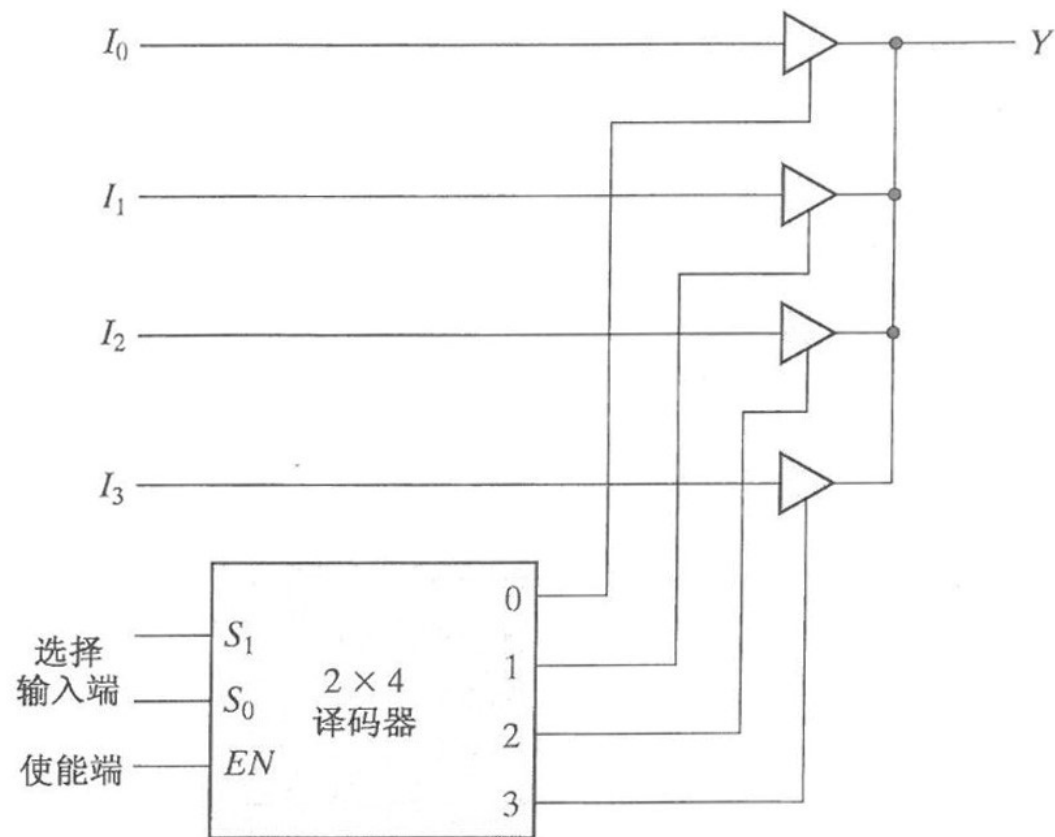
三态缓冲器图形符号



数据选择器

■ 三态门

● 例：三态门实现的四选一数据选择器



组合逻辑电路设计

- 一. 简介
- 二. 比较器
- 三. 译码器
- 四. 编码器
- 五. 多路选择器
- 六. 加法器

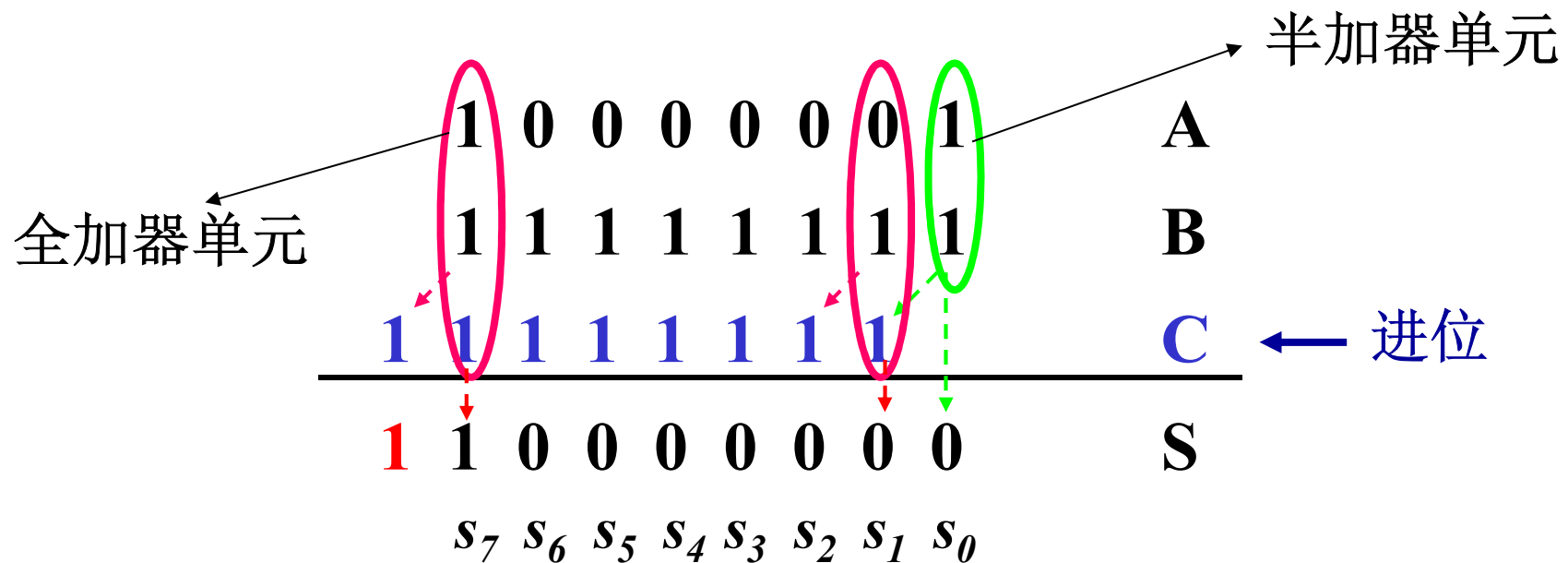
二进制加法器、BCD加法器、减法器

加法器 (Adder)

功能: $S = A + B$

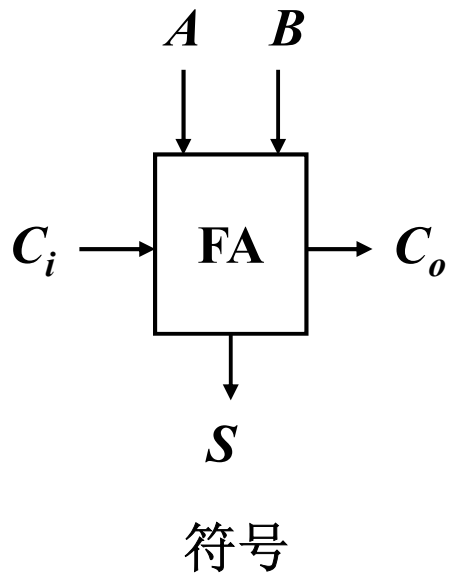
例如: $A = 129$, $B = 255$, 则 $S = 384$

二进制表示: $A = 10000001$, $B = 11111111$, $S = 110000000$



全加器 (Full Adder)

真值表



C_i	A	B	S	C_o	$P=A+B$	$G=AB$	$A\oplus B$
0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0
0	0	1	1	0	1	0	1
1	0	1	0	1	1	0	1
0	1	0	1	0	1	0	1
1	1	0	0	1	1	0	1
0	1	1	0	1	1	1	0
1	1	1	1	1	1	1	0

全加器 (Full Adder)

和 (S) 与进位 (C_o)

$$S = A \oplus B \oplus C_i = \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i + ABC_i$$

$$C_o = AB + BC_i + AC_i$$

中间信号

$$G = AB$$

进位产生 (**Generate**)

$$D = \overline{A}\overline{B}$$

进位取消 (**Delete**)

$$P = A + B \text{ 或 } (P = A \oplus B)$$

进位传播 (**Propagate**)

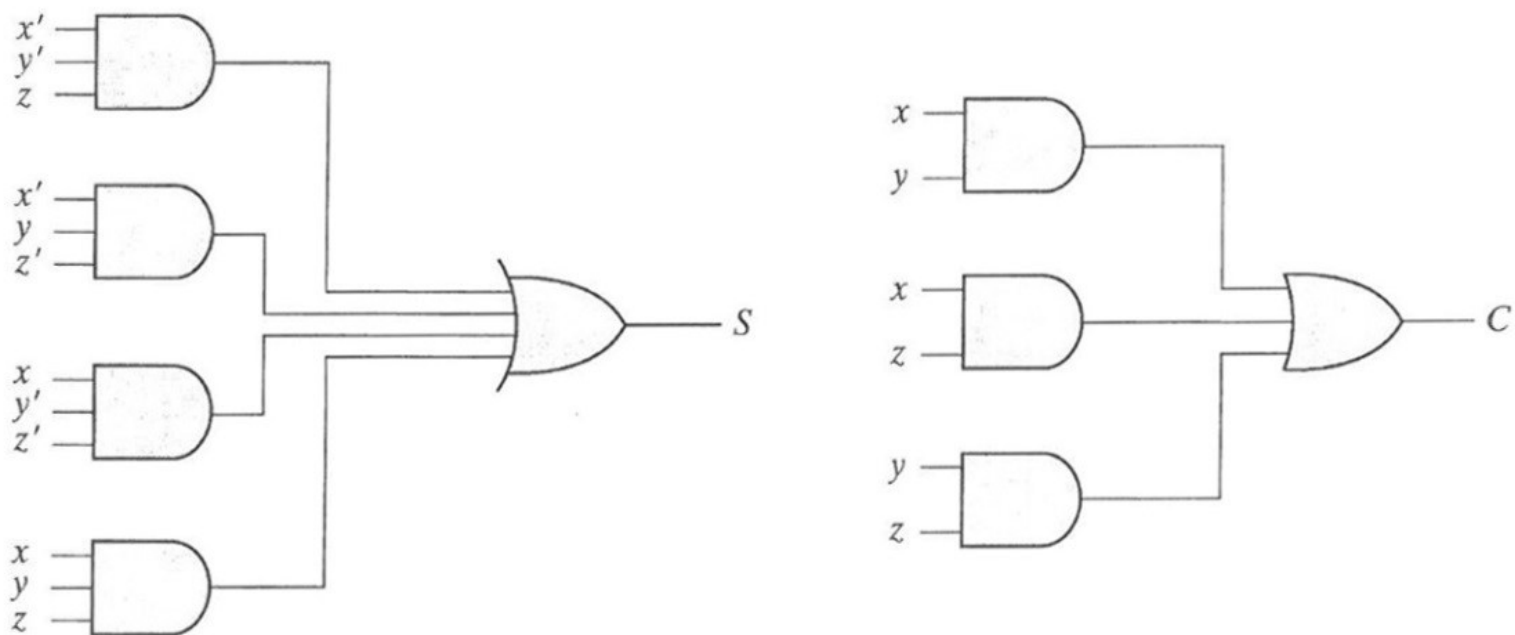
用中间信号表示的S和C_o

$$C_o(G, P) = G + PC_i$$

$$S(G, P) = P \oplus C_i$$

全加器 (Full Adder)

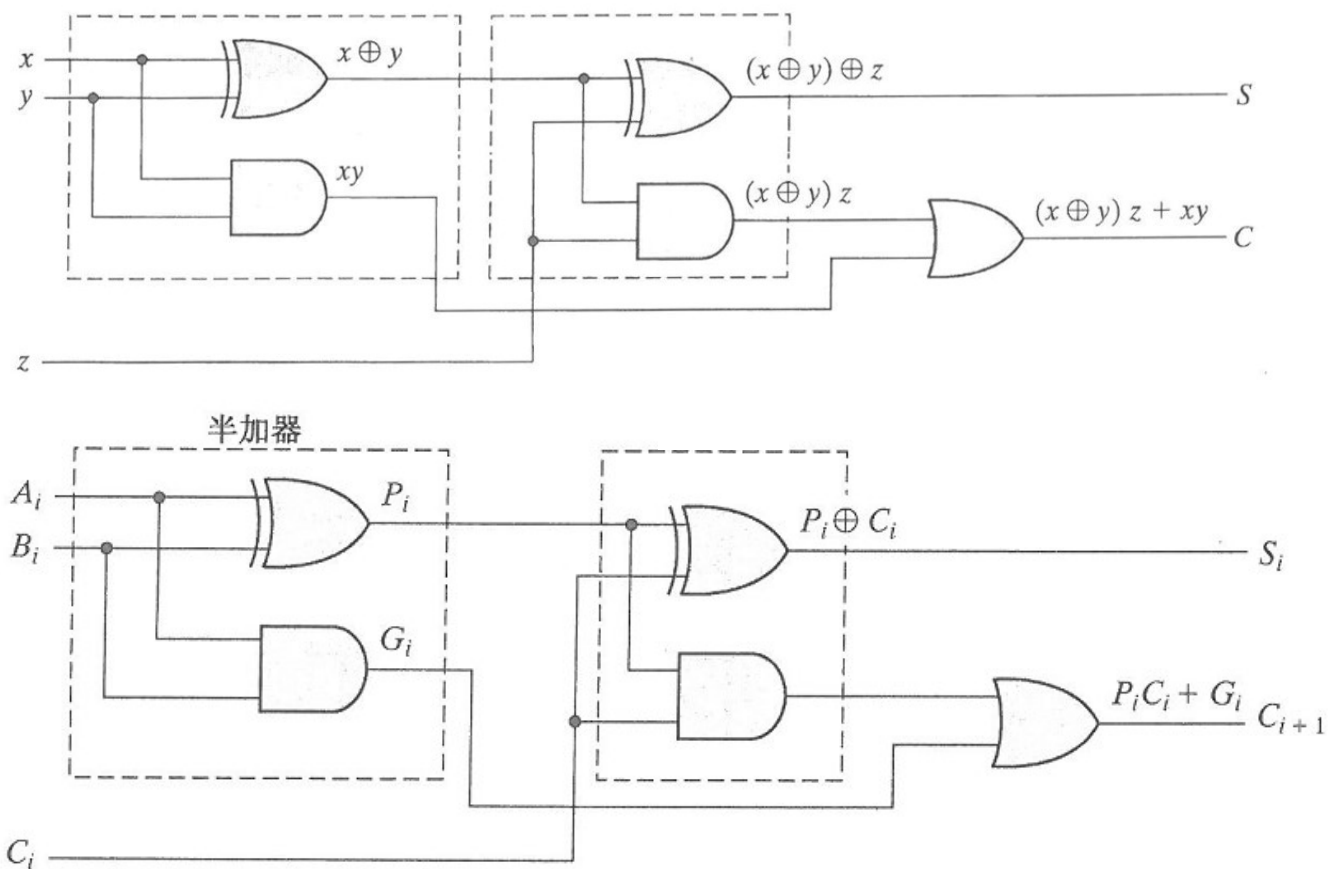
■ 全加器逻辑电路



与-或逻辑电路 (积之和式)

全加器 (Full Adder)

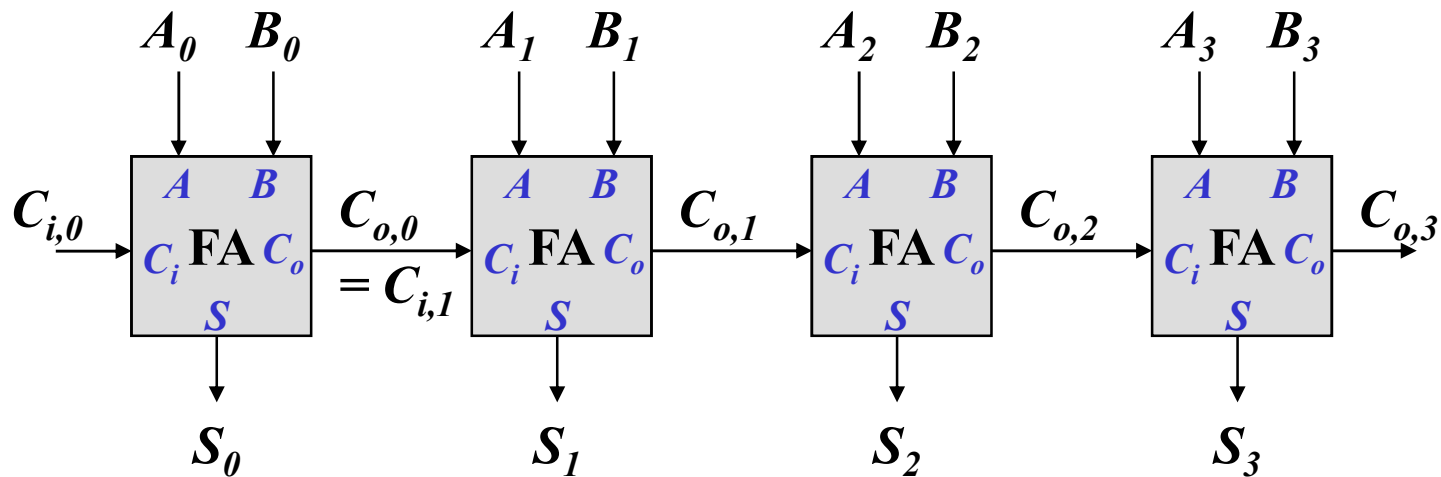
■ 全加器逻辑电路



两个半加器实现全加器

逐位进位（ripple-carry）加法器

4位逐位进位加法器



最坏情况下加法器的延时: $t_{adder} \approx (N-1)t_{carry} + t_{sum}$

- $t_{adder} \propto N$
- t_{carry} 比 t_{sum} 对加法器延时影响更大, 优化 t_{carry} 比 t_{sum} 更重要

加法器 (Adder)

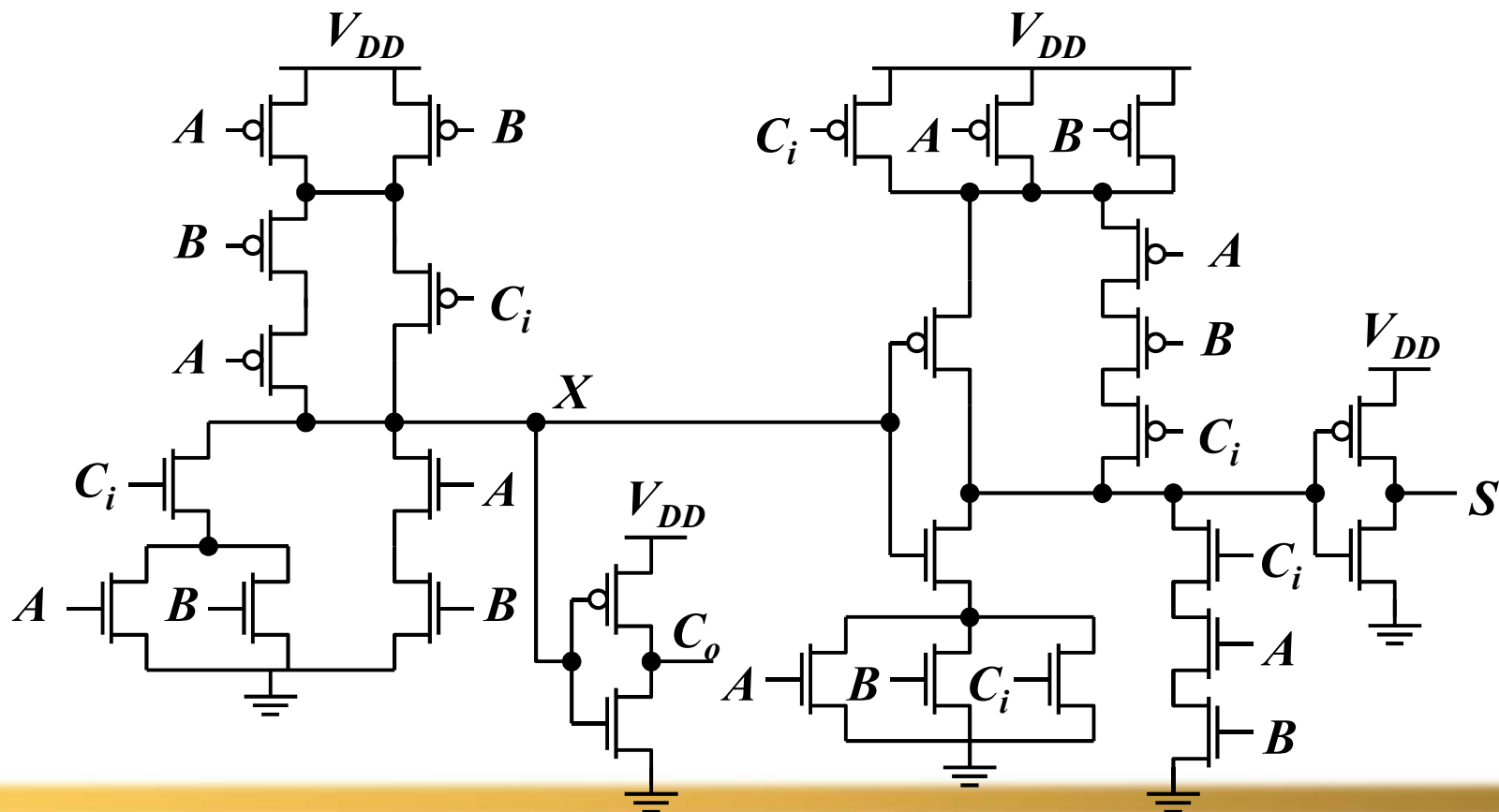
- 两个层次的优化（面积与速度）：
 - 电路级：晶体管尺寸、拓扑结构
 - 逻辑级：重新排列布尔表达式

全加器电路设计

静态互补CMOS全加器电路

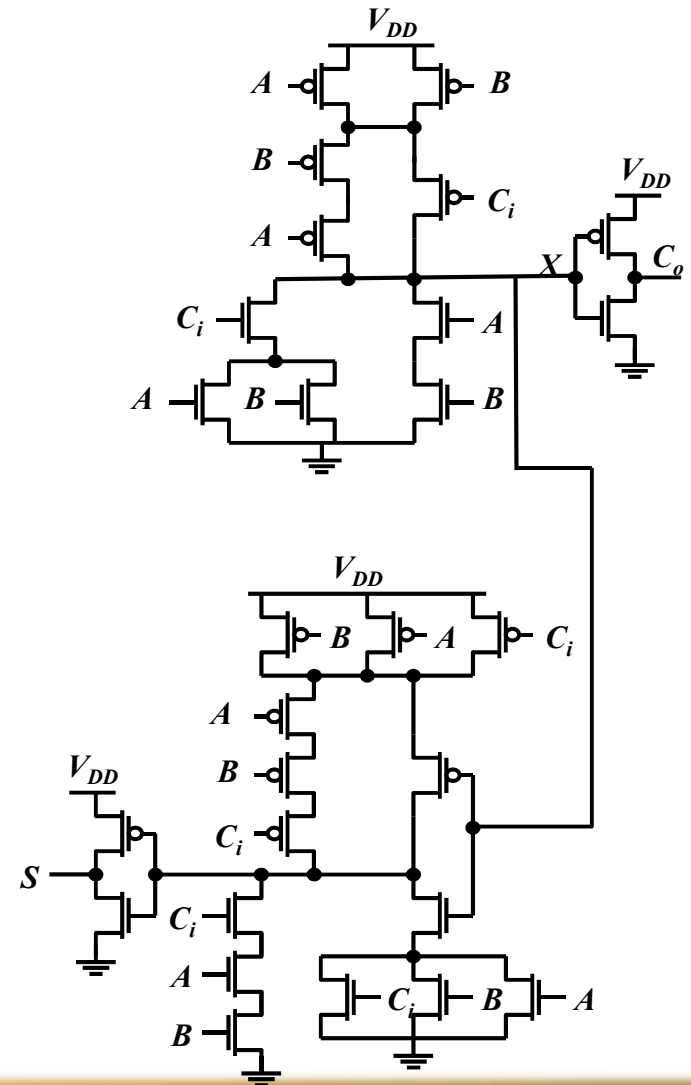
$$C_o = AB + BC_i + AC_i$$

$$S = A\bar{B}\bar{C}_i + \bar{A}B\bar{C}_i + \bar{A}\bar{B}C_i + ABC_i = ABC_i + \bar{C}_o(A + B + C_i)$$



静态互补CMOS全加器特点

- 面积较大（**28**个晶体管）
- 速度较慢
 - 进位产生及求和电路存在多个**PMOS**管串联
 - 信号 C_o 负载电容较大：**2**个扩散电容和**6**个栅电容
 - 进位产生电路有两级反相
 - 和产生电路有**1**个额外反相器
- 与 C_i 相连接的晶体管位置
 - **PMOS**管放置在串联管子少的路径上
 - 所有晶体管放置尽量靠近输出节点

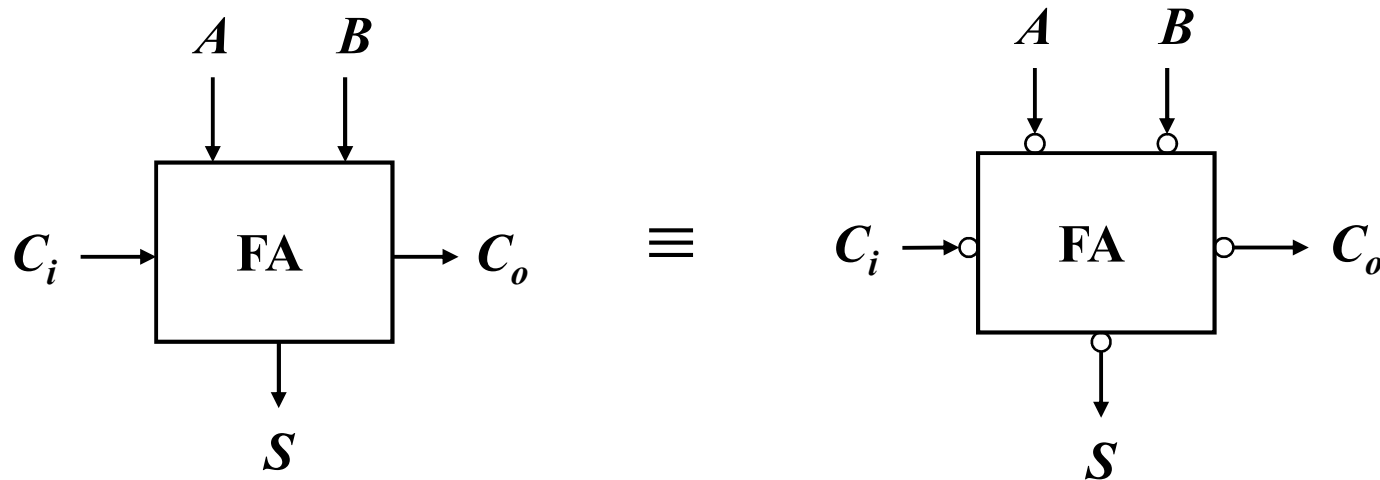


加法器的反相特性

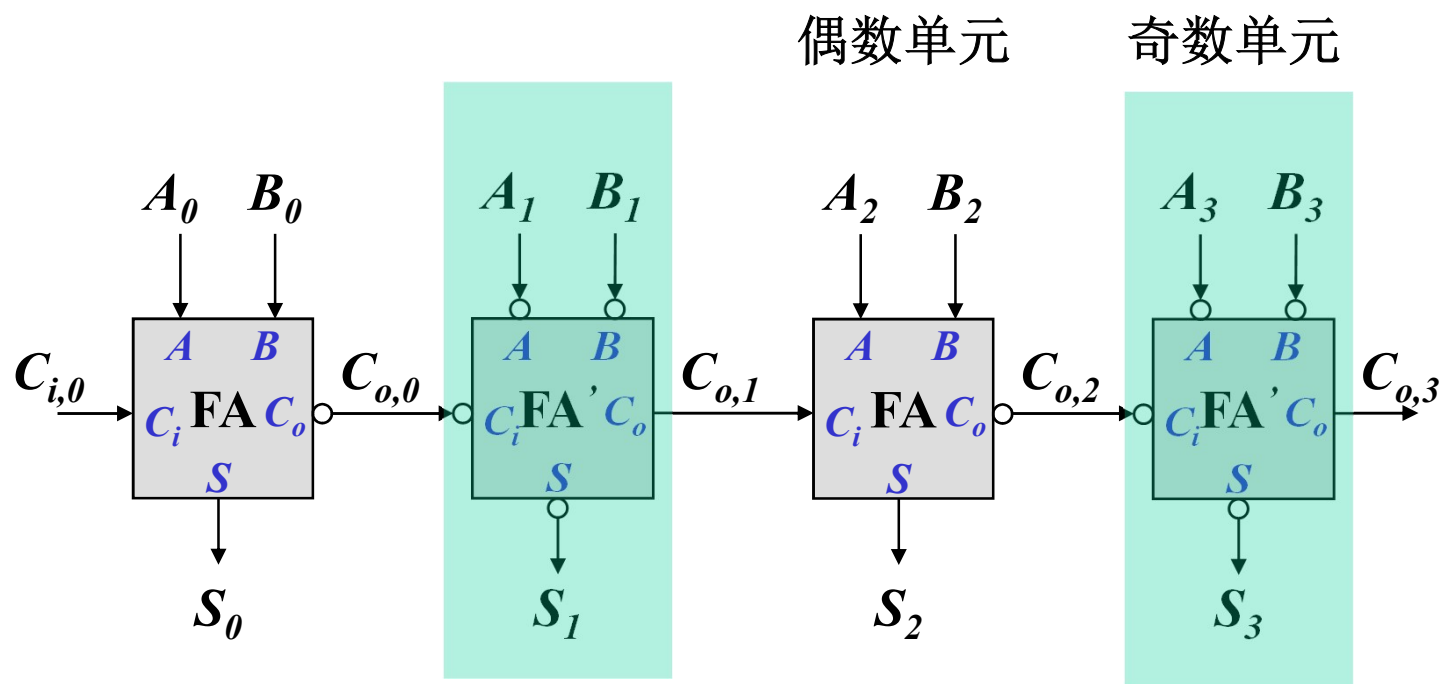
全加器所有输入反相，它的所有输出也反相

$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i)$$

$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i)$$

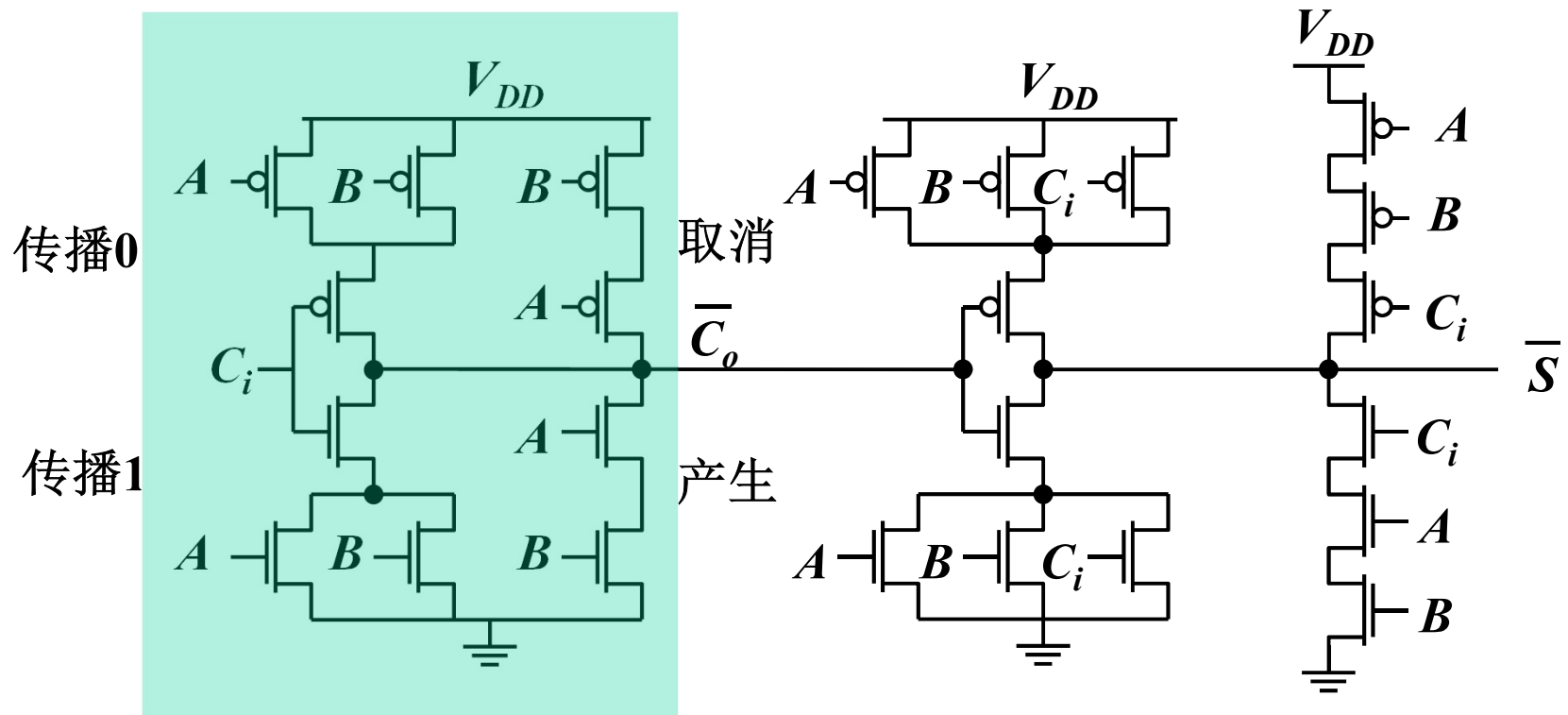


利用反相特性提高速度



偶数和奇数位采用不同的全加器单元

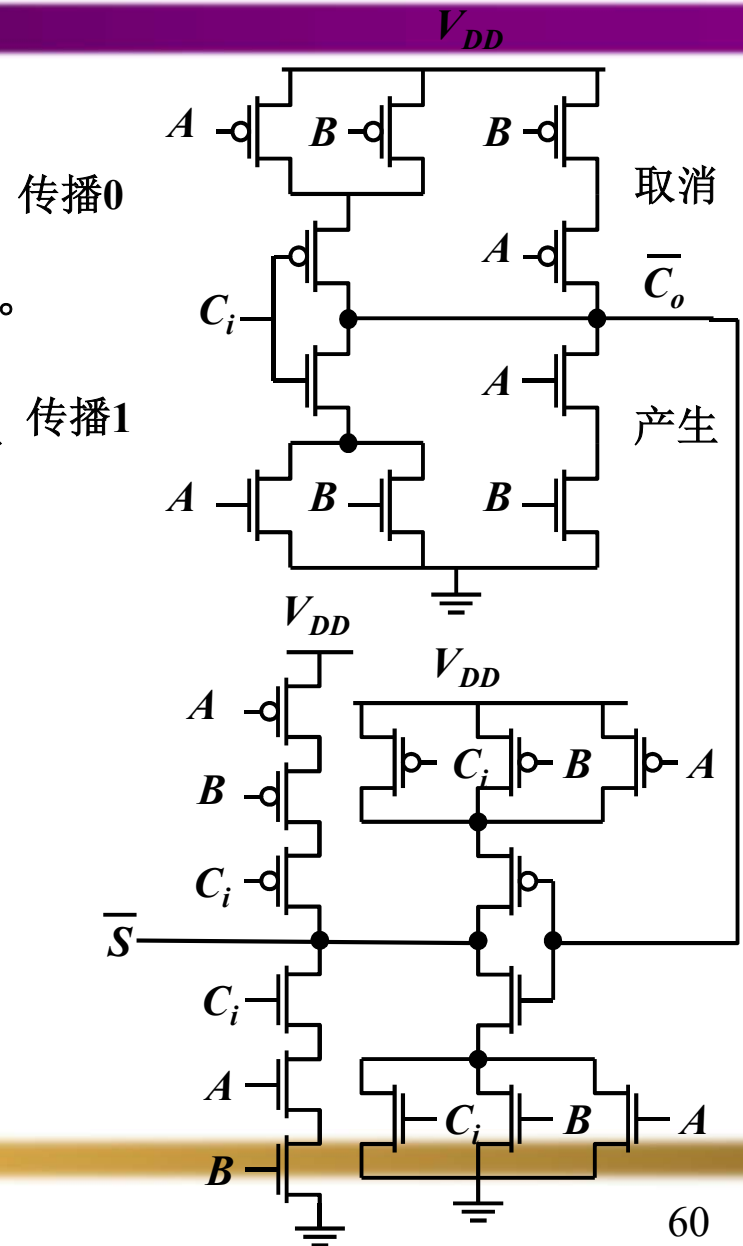
镜像加法器 (Mirror Adder)



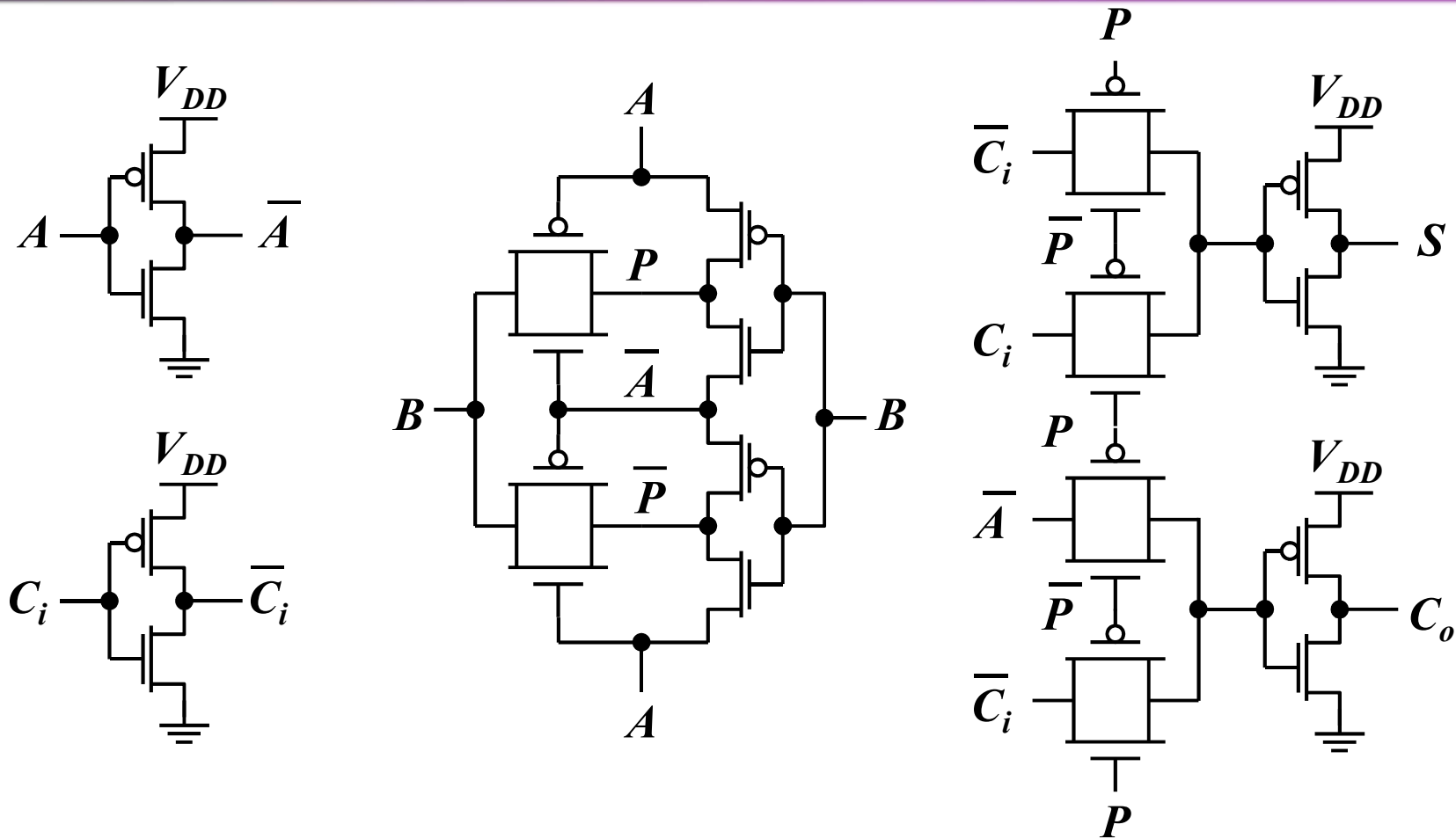
- 消除了进位链上的反相器
- PDN和PUN不是对偶的

镜像加法器特点

- 24个晶体管
- NMOS与PMOS链完全对称，选择合适的器件尺寸，可保证相同的上升和下降时间。进位产生电路最多两个晶体管串联。
- 版图设计时，最关键的问题是使 C_0 节点的电容最小。减少扩散电容是最重要的。
- 节点 C_0 的电容由4个扩散电容、两个内部栅电容和6个负载栅电容。
- 接 C_i 的晶体管放置在最靠近输出的位置。
- 只有进位链上的晶体管尺寸需要优化以得到最快的速度。求和级上的所有晶体管可用最小尺寸晶体管。



传输门加法器



和 (**sum**) 与进位 (**carry**) 具有相同的延时

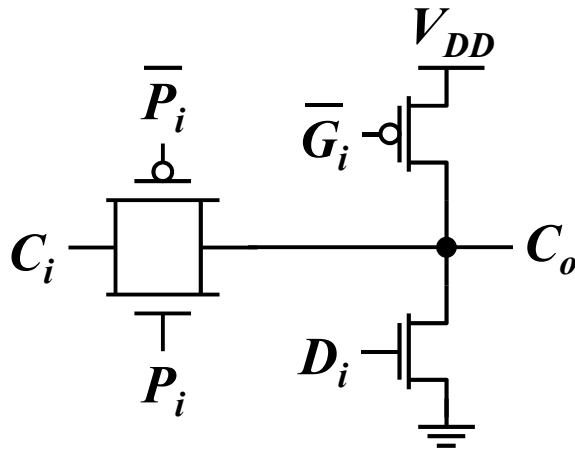
全加器电路设计小结

■ 电路级的优化方法小结

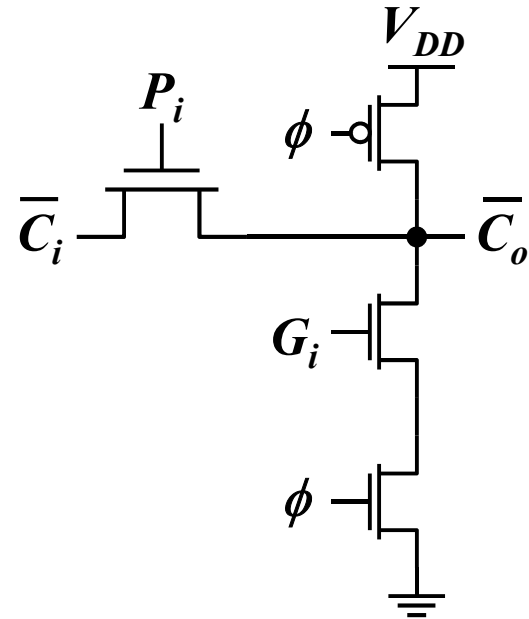
- 加大电路中进位级的晶体管尺寸提高进位输出的驱动能力
- 减小进位信号所驱动的晶体管尺寸，即减小求和级中的晶体管尺寸，以减小进位信号的负载
- 尽量使进位信号的连线短，减小连线寄生电容和寄生电阻
- 把进位输入信号所驱动的晶体管尽量放置在靠近输出端的位置
- 详细的电路模拟确定晶体管尺寸
- 利用加法器的反相特性，进行结构优化

Manchester进位链

Manchester进位门



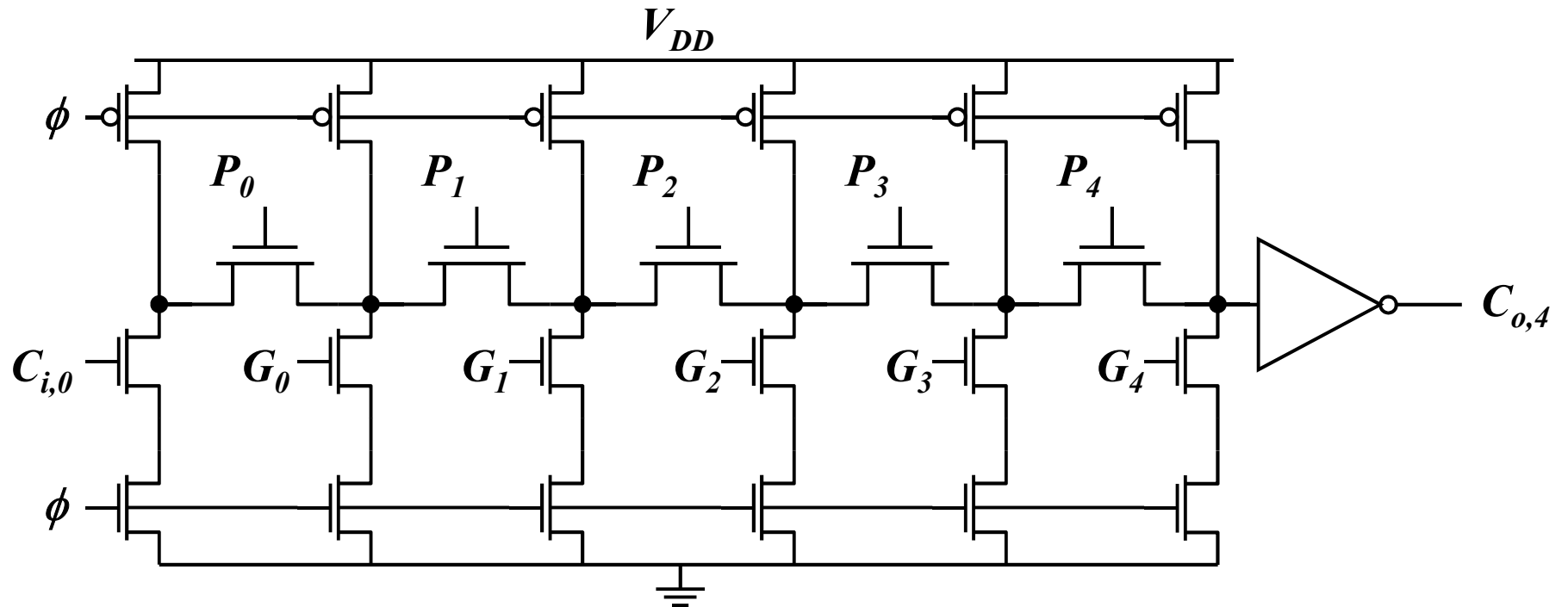
(a) 静态实现



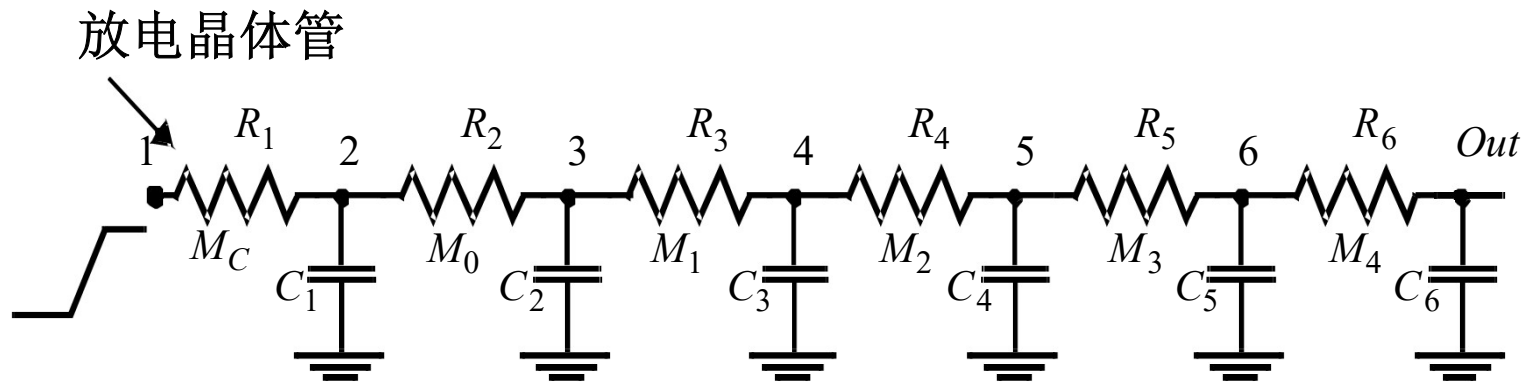
(b) 动态实现

Manchester进位链

动态逻辑实现的Manchester进位链（5比特）



Manchester进位链尺寸设计



$$t_p = 0.69 \sum_{i=1}^N C_i \left(\sum_{j=1}^i R_j \right)$$
$$= 0.69 \frac{N(N+1)}{2} RC$$

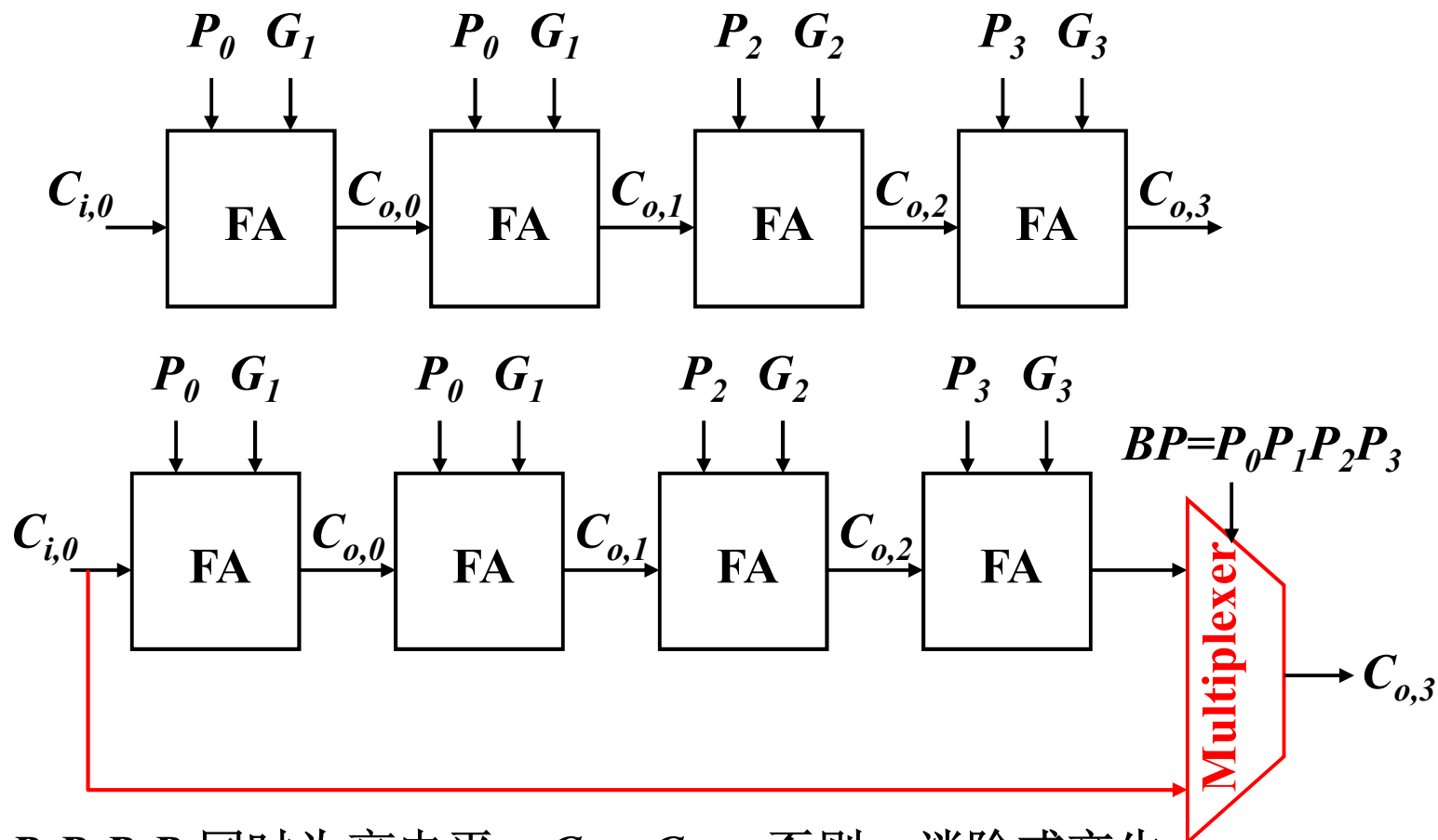
延时与级数 N 的关系:

$$t_p \propto N^2$$

插入信号缓冲反相器

加法器的逻辑优化

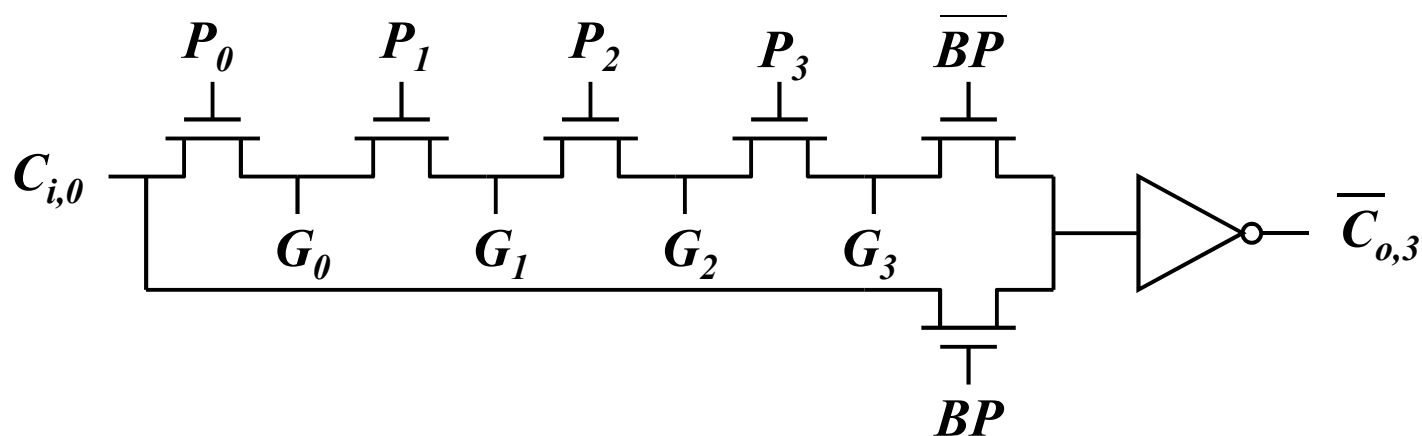
■ 进位旁路（Carry-Bypass）加法器



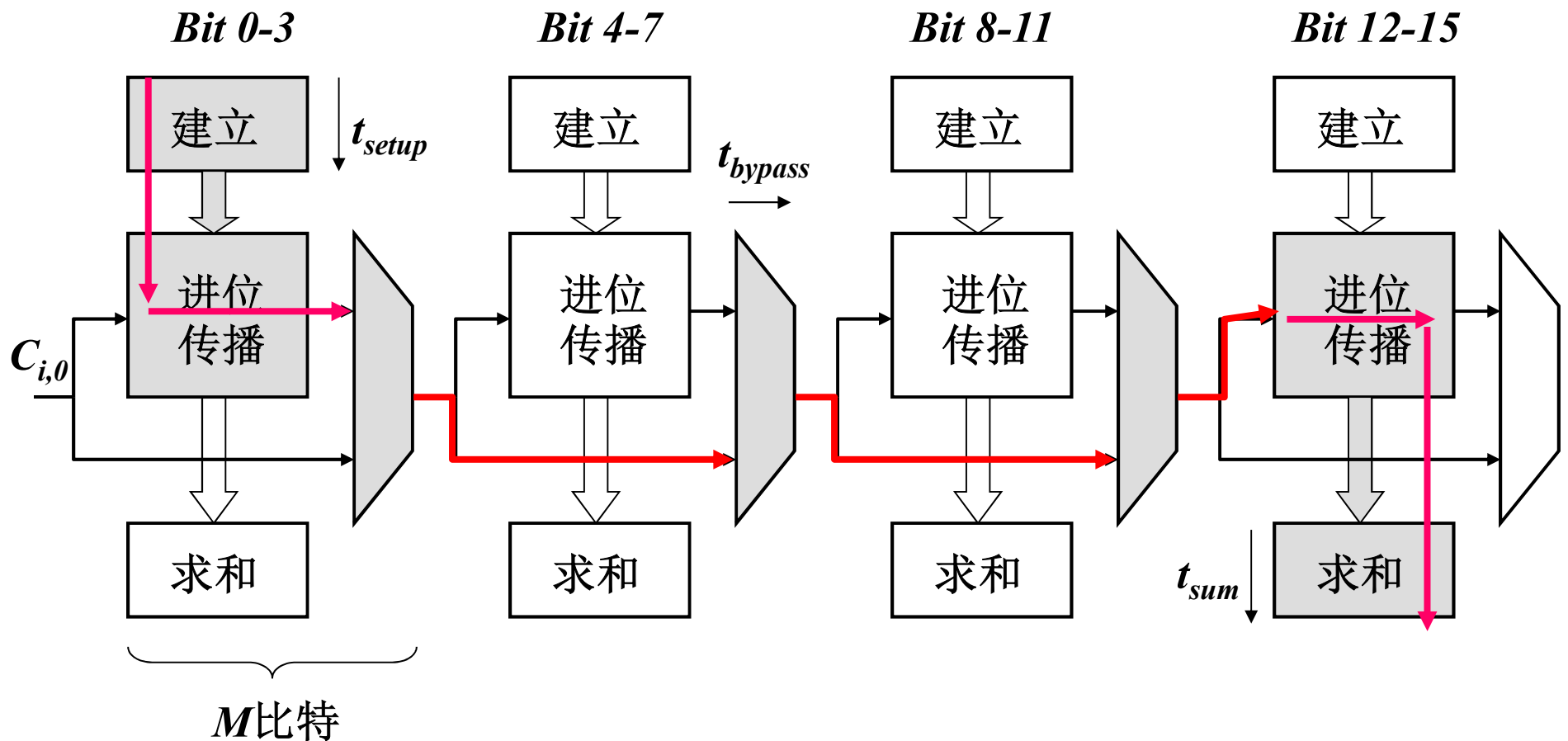
如果 $P_0P_1P_2P_3$ 同时为高电平， $C_{o,3}=C_{i,0}$ ；否则，消除或产生

进位旁路加法器

■ Manchester进位链实现旁路加法器



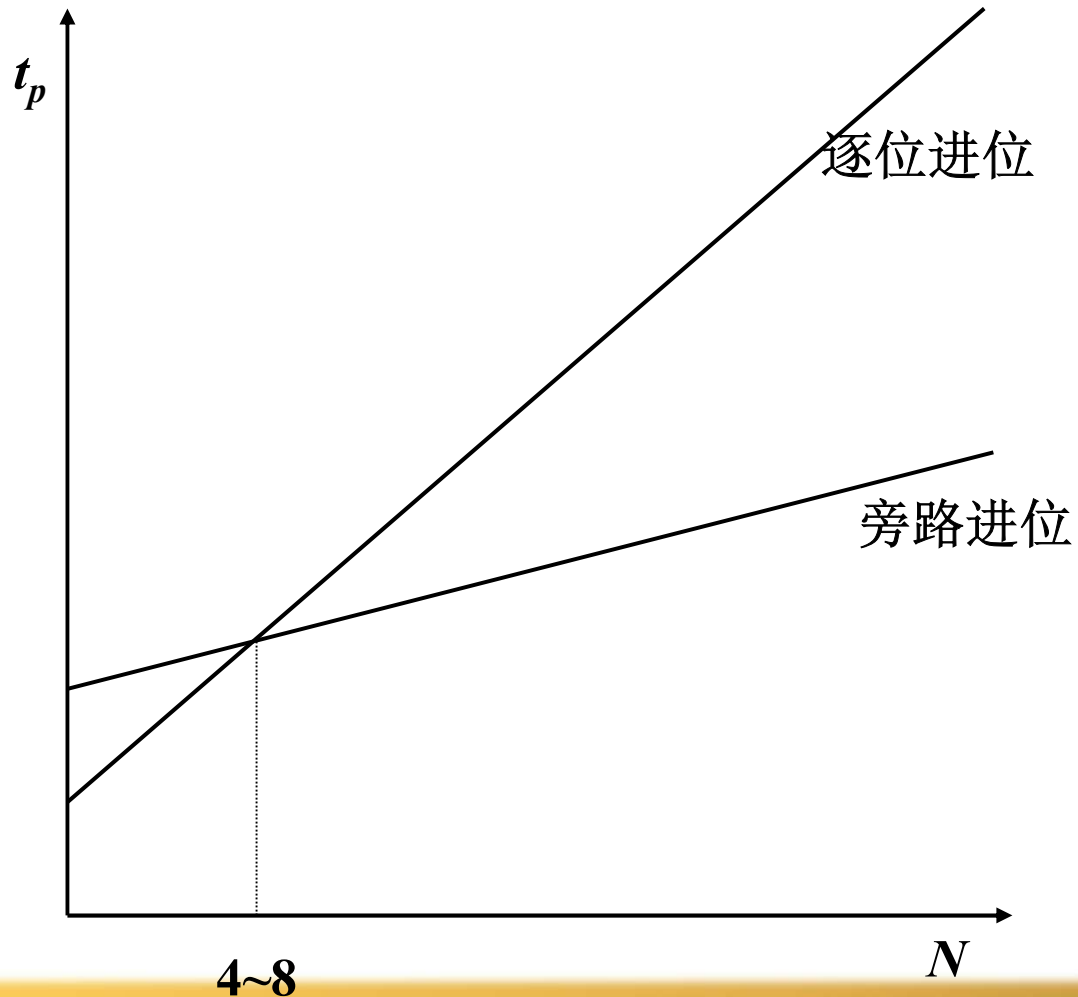
进位旁路加法器



$$t_{adder} = t_{setup} + Mt_{carry} + (N/M - 1)t_{bypass} + (M - 1)t_{carry} + t_{sum}$$

进位旁路加法器

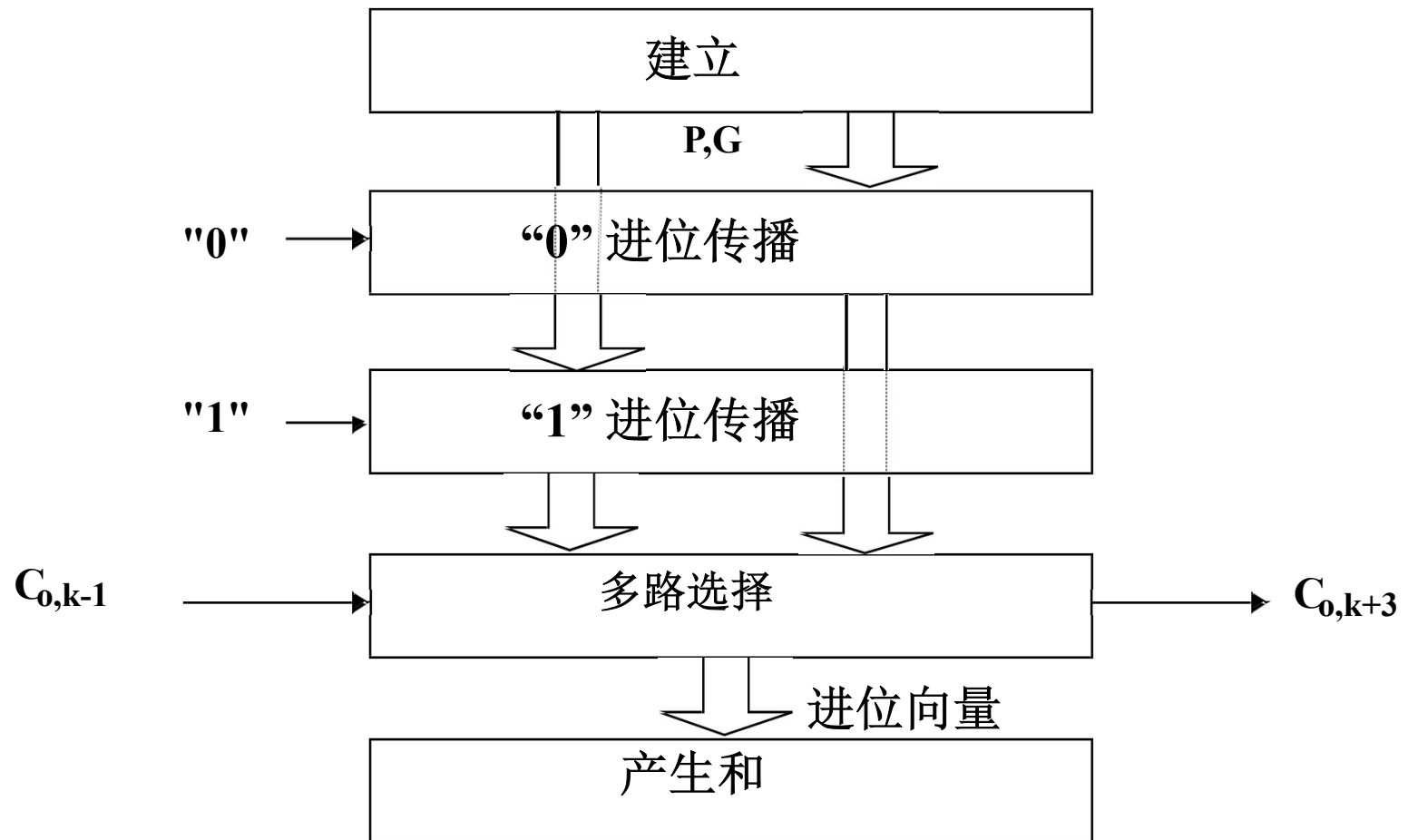
■ 逐位进位与进位旁路性能比较



- 进位旁路加法器的总进位传播时间仍与级数 N 成正比，但比例系数比逐位进位小
- N 较小时，旁路的额外开销使得延时的改善不大，一般 $N > 4\sim 8$ 时才采用进位旁路
- 面积会略微增加。
10%~20%

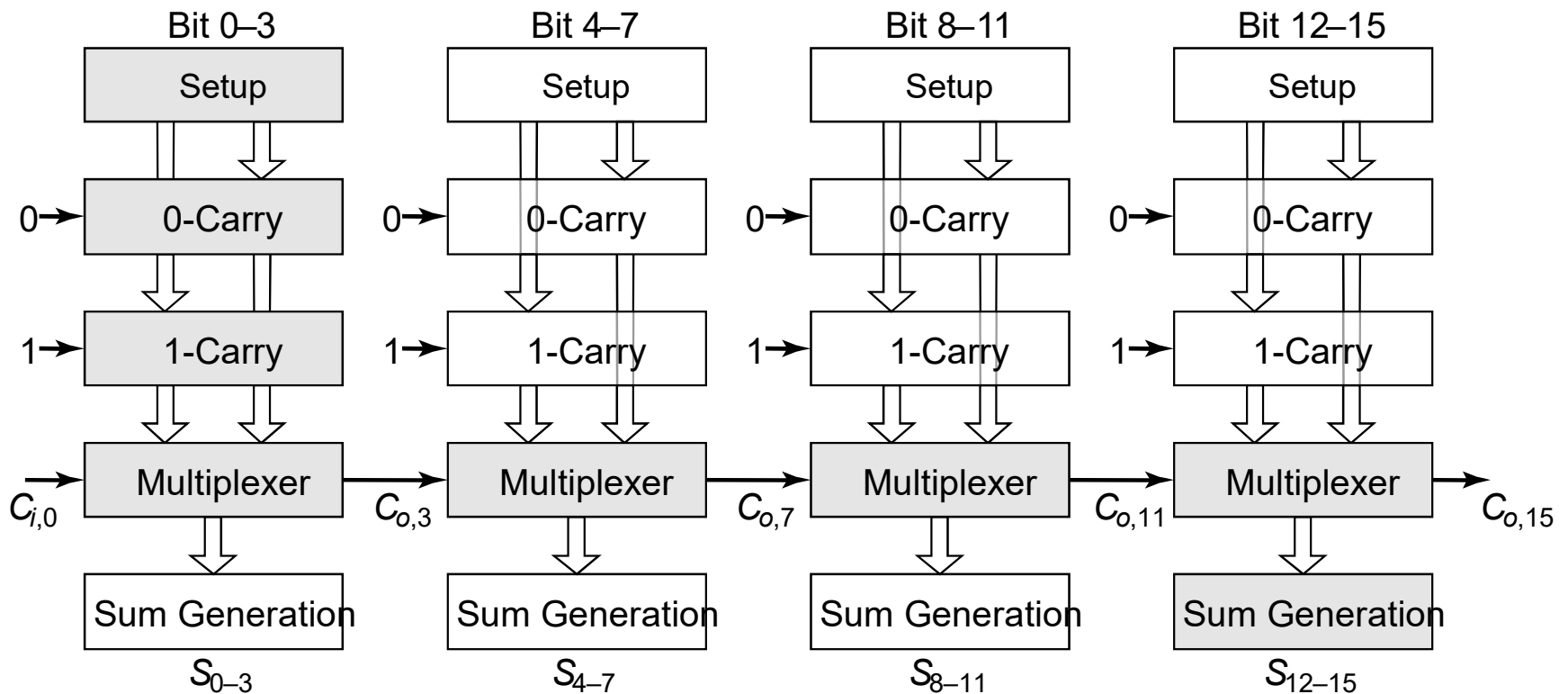
线性进位选择加法器

■ 4位选择进位模块



线性进位选择加法器

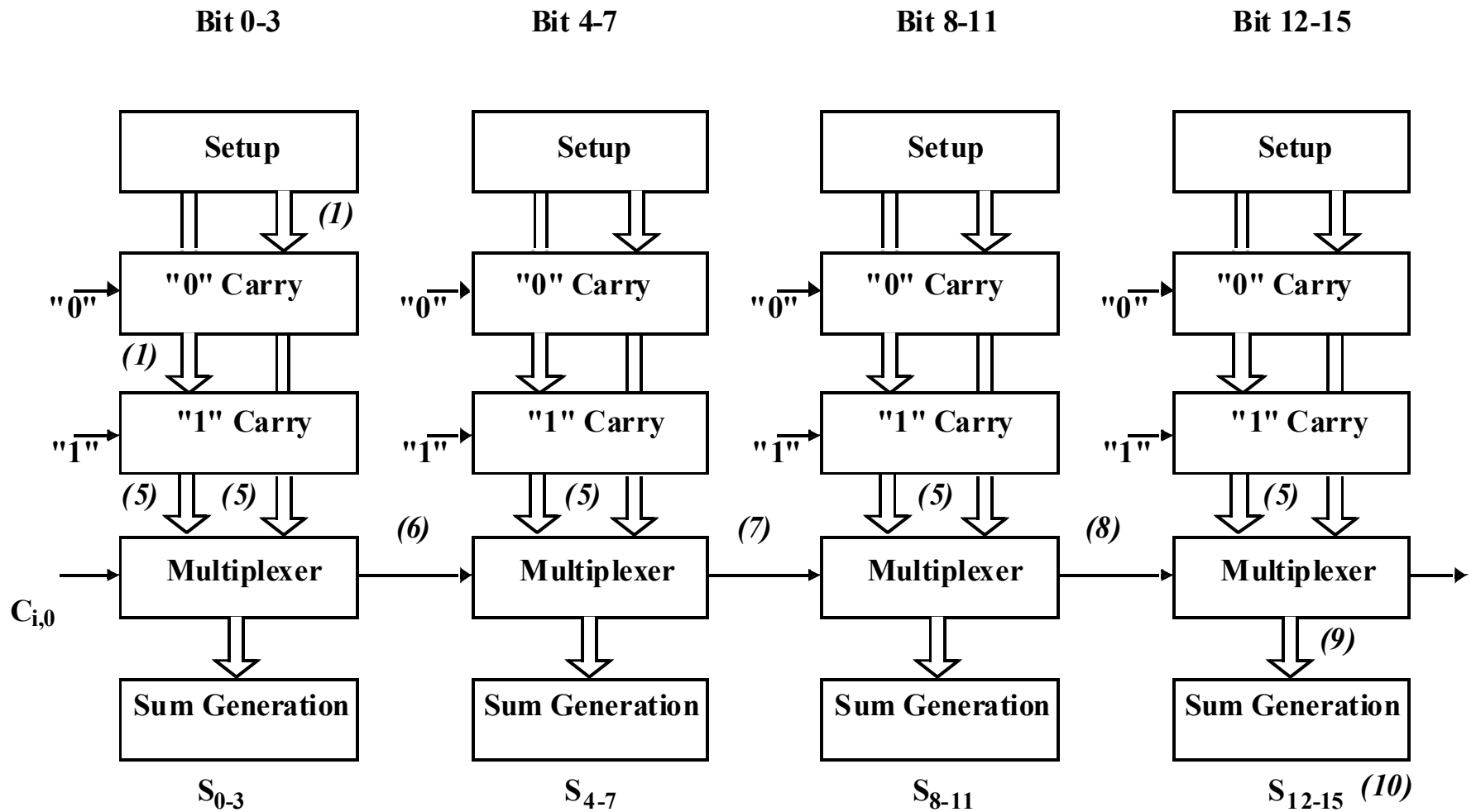
■ 关键路径（Critical Path）



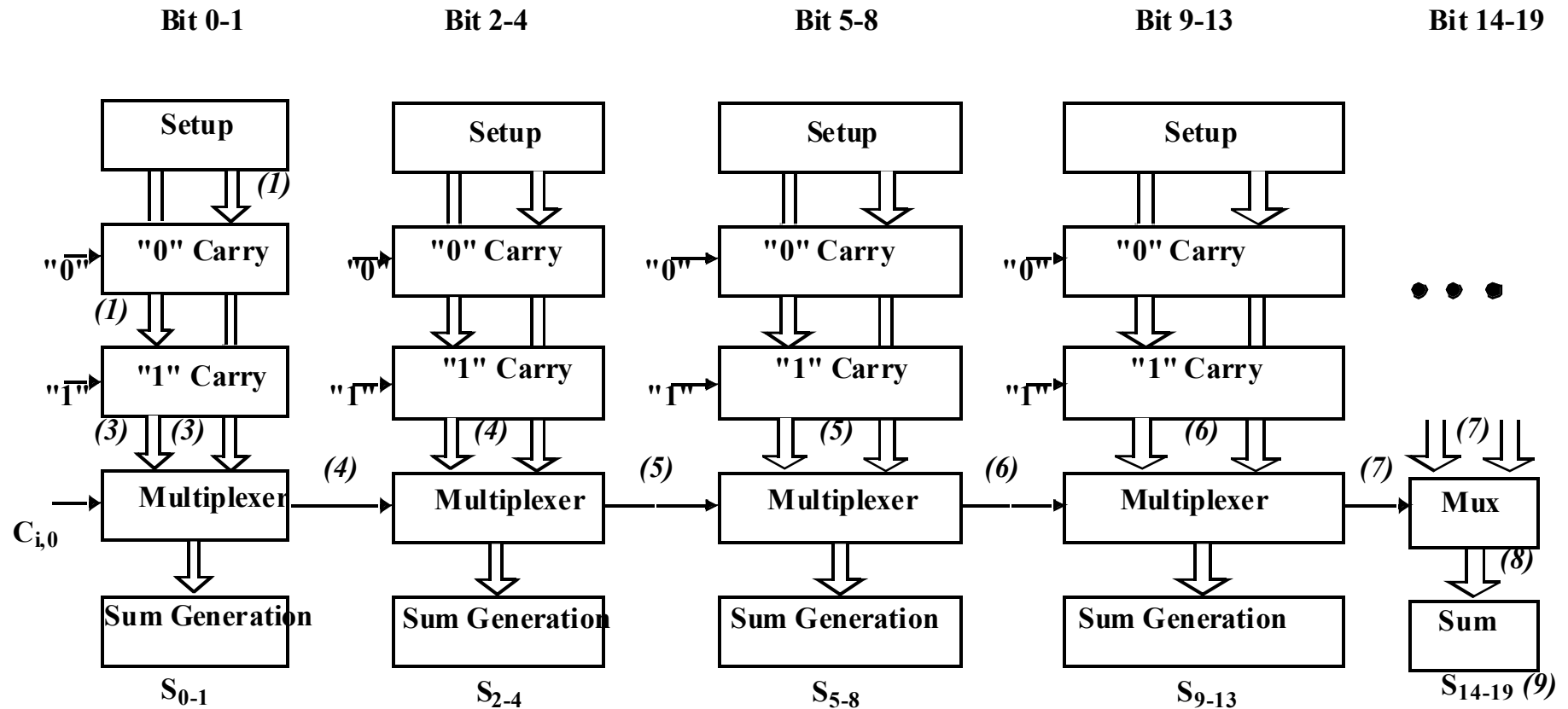
$$t_{adder} = t_{setup} + Mt_{carry} + \frac{N}{M}t_{mux} + t_{sum}$$

线性进位选择加法器

■ 最坏情况信号到达时间分析

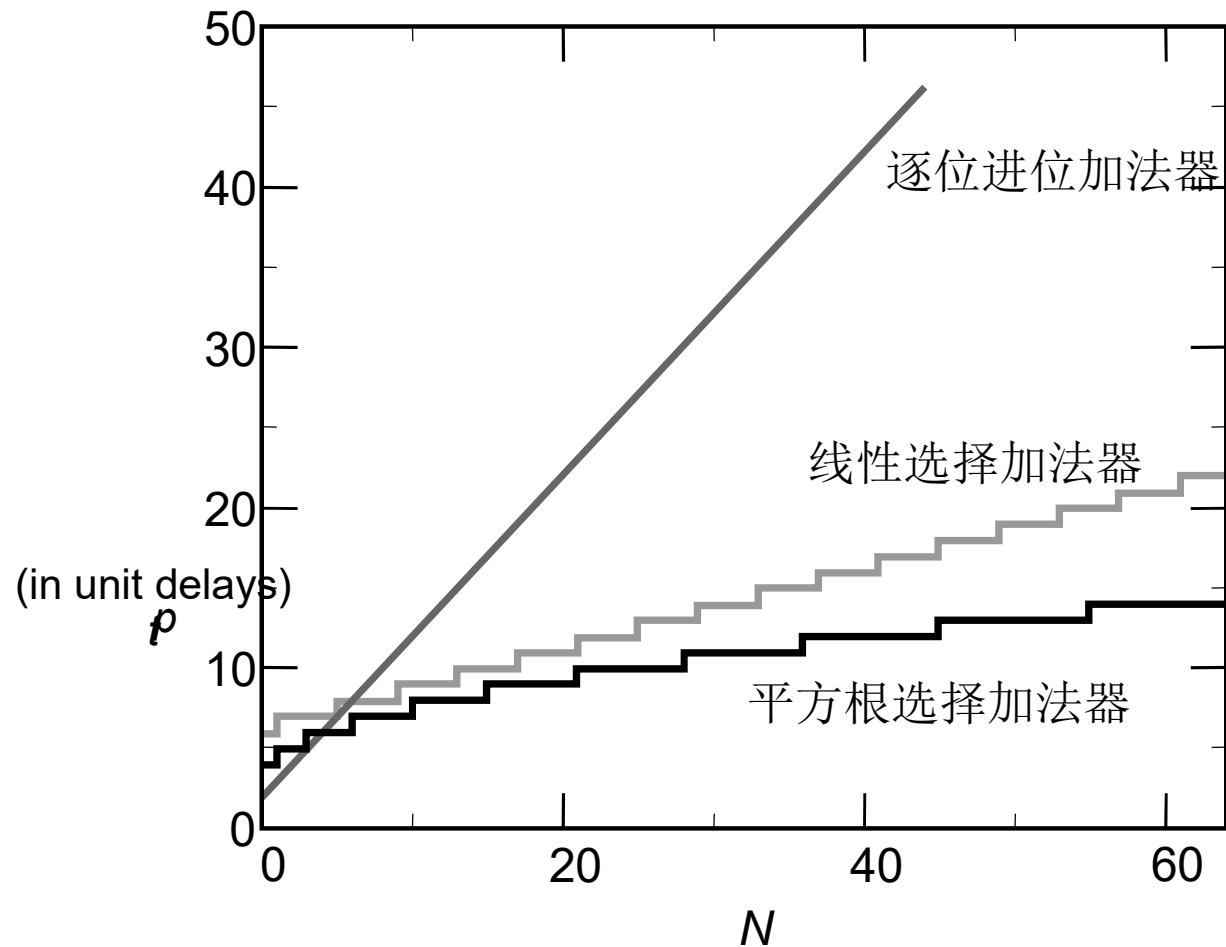


平方根进位选择加法器



$$t_{adder} = t_{setup} + Mt_{carry} + \sqrt{2N}t_{mux} + t_{sum}$$

进位选择加法器的延时比较



超前进位加法器

■ 超前进位（Carry-Lookahead）

第*i*位产生的进位 c_{i+1} : $c_{i+1} = g_i + p_i c_i$

第*i+1*位产生的进位 c_{i+2} : $c_{i+2} = g_{i+1} + p_{i+1} c_{i+1}$
 $= g_{i+1} + p_{i+1} g_i + p_{i+1} p_i c_i$

输出进位 c_{i+2} :

- 在第*i+1*级上产生 (g_{i+1})
- 第*i*级产生, 第*i+1*级传播 ($p_{i+1} g_i$)
- 第*i*级和第*i+1*级传播 ($p_{i+1} p_i c_i$)

超前进位加法器

$$C_{o,k} = f(A_k, B_k, C_{o,k-1}) = G_k + P_k C_{o,k-1}$$

$$\begin{aligned} C_{o,k} &= G_k + P_k C_{o,k-1} = G_k + P_k (G_{k-1} + P_{k-1} C_{o,k-2}) \\ &= G_k + P_k (G_{k-1} + P_{k-1} (\dots + P_1 (G_0 + P_0 C_{i,0}))) \end{aligned}$$

$$C_{o,0} = G_0 + P_0 C_{i,0}$$

$$C_{o,1} = G_1 + P_1 C_{o,0} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0}$$

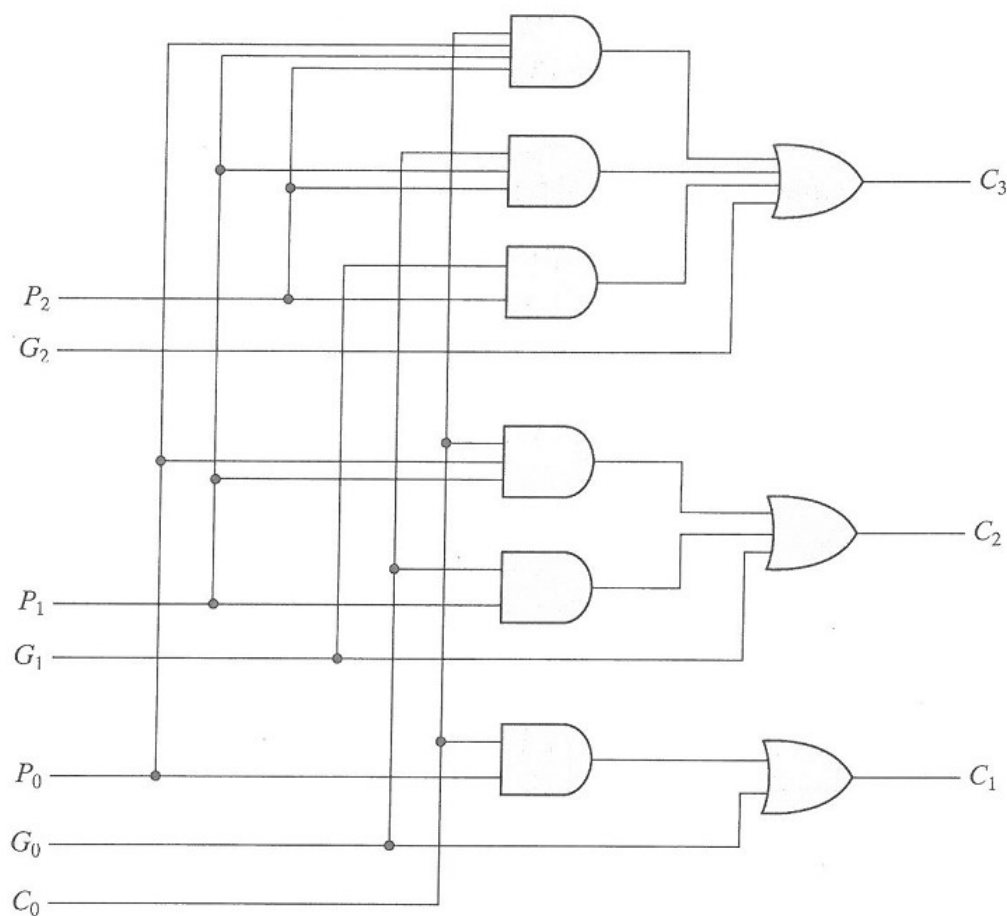
$$C_{o,2} = G_2 + P_2 C_{o,1} = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{i,0}$$

$$C_{o,3} = G_3 + P_3 C_{o,2} = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{i,0}$$

⋮

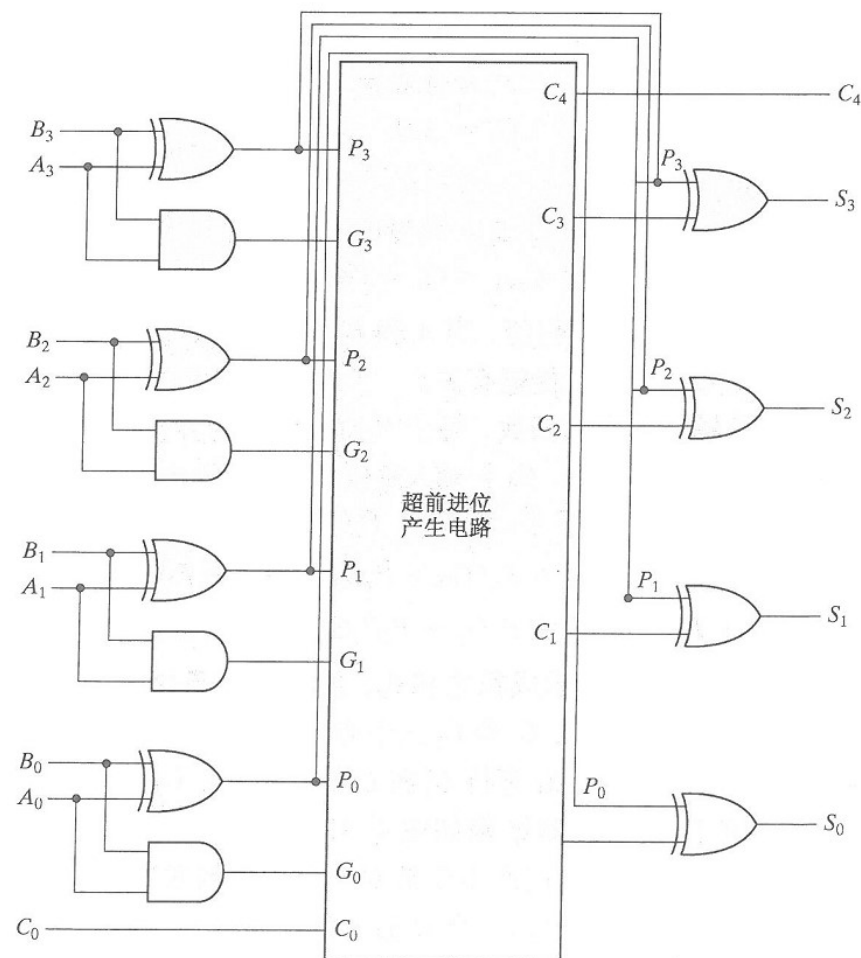
超前进位加法器

■ 超前进位产生的电路逻辑图

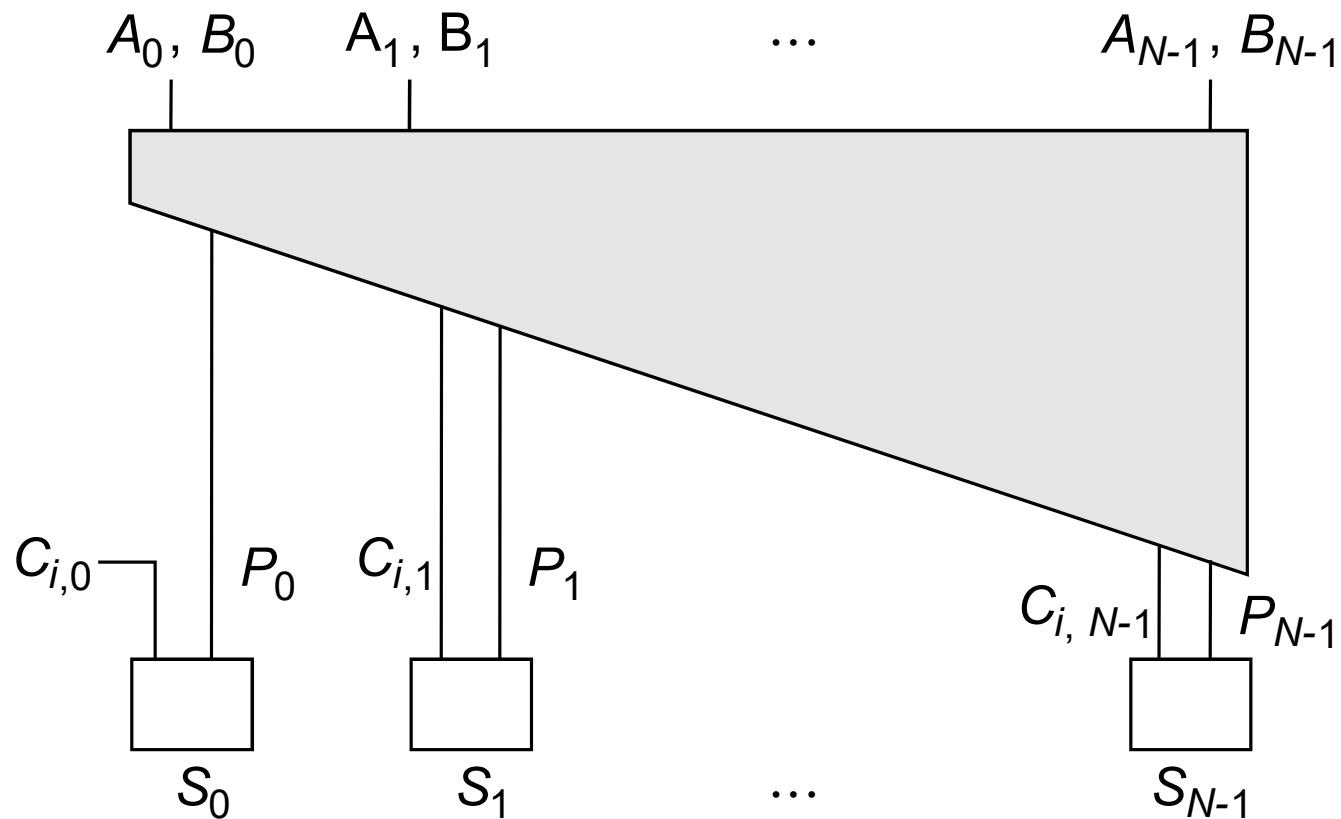


超前进位加法器

■ 带超前进位的4位加法器



超前进位加法器



从逻辑设计角度讲，**SUM** 的求解
不再与位数 N 相关

超前进位加法器

➤ 从逻辑设计角度

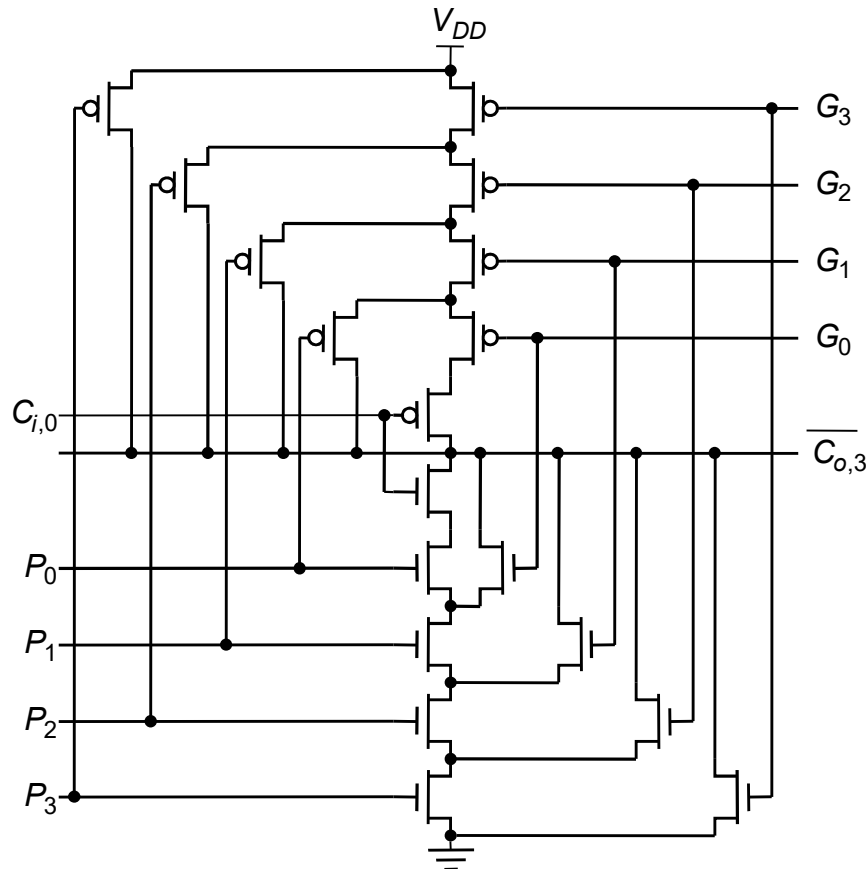
超前进位加法器的进位递归式的展开可以一直继续下去，如果展开 k 次，就得到两个层次的 $AND-OR$ 结构，每一位的进位输出只与最初的进位输入有关，而与比它低位的其他进位输入无关，因此加法时间与位数 N 无关。

➤ 从电路设计角度

AND 的扇入为 $N+1$ ， OR 的扇入为 $N+1$ ，于是有 $N+1$ 个 MOS 晶体管堆叠。因此，当位数 N 较大时，电路的大扇入会使速度变慢。而且某些信号的扇出还有可能进一步地增加，使加法器的延时加大。同时，加法器的实现面积也会随位数 N 的增加而增加。因此这一超前进位只对小的 N 值（ $N \leq 4$ ）有效。

超前进位加法器

■ 超前进位结构



4位超前进位电路图

级数 N 较大时:

- 扇入 (FI) 大
- 扇出 (FO) 大
- 面积随 N 增加而显著增加

级数 $N \leq 4$

对数超前进位加法器

- **N**位超前进位组的晶体管实现有**N+1**个并行分支，并且最多有**N+1**个晶体管堆叠，因此超前进位计算在实际中只限于**N≤4**
- 建立快速的加法器可以把进位传播和进位产生组织成递归的树形结构，进位传播进行层次化分解

$$C_{o,0} = G_0 + P_0 C_{i,0}$$

$$\begin{aligned} C_{o,1} &= G_1 + P_1 C_{o,0} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0} = (G_1 + P_1 G_0) + (P_1 P_0) C_{i,0} \\ &= G_{1:0} + P_{1:0} C_{i,0} \end{aligned}$$

$$C_{o,2} = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{i,0} = G_2 + P_2 C_{o,1}$$

$$\begin{aligned} C_{o,3} &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{i,0} \\ &= (G_3 + P_3 G_2) + (P_3 P_2) C_{o,1} = G_{3:2} + P_{3:2} C_{o,1} \end{aligned}$$

对数超前进位加法器

- 把产生和传播函数看作一对 $(\mathbf{G}, \mathbf{P})$ ，并定义点操作运算 $(\bullet)$

$$(G, P) \bullet (G', P') = (G + PG', PP')$$

- 点操作运算满足结合律，不满足交换律

$$(G, P) \bullet (G', P') \bullet (G'', P'') = (G, P) \bullet ((G', P') \bullet (G'', P''))$$

$$(G, P) \bullet (G', P') \bullet (G'', P'') \neq (G, P) \bullet (G'', P'') \bullet (G', P')$$

对数超前进位加法器

➤ 用点操作表示的4位加法器

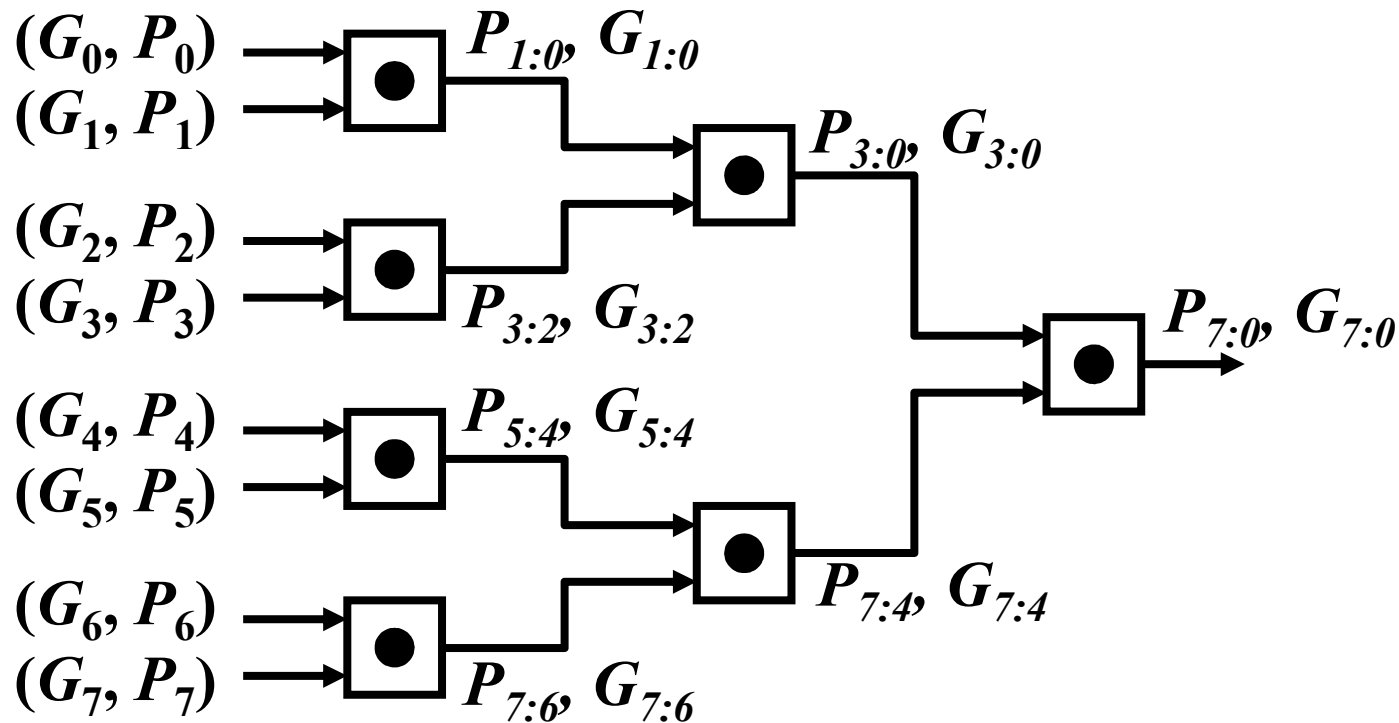
$$(C_{o,0},0) = (G_0, P_0) \bullet (C_{i,0},0)$$

$$(C_{o,1},0) = (G_1, P_1) \bullet (G_0, P_0) \bullet (C_{i,0},0) = (G_{1:0}, P_{1:0}) \bullet (C_{i,0},0)$$

$$\begin{aligned}(C_{o,2},0) &= (G_2, P_2) \bullet (G_1, P_1) \bullet (G_0, P_0) \bullet (C_{i,0},0) \\ &= (G_{2:1}, P_{2:1}) \bullet (G_0, P_0) \bullet (C_{i,0},0)\end{aligned}$$

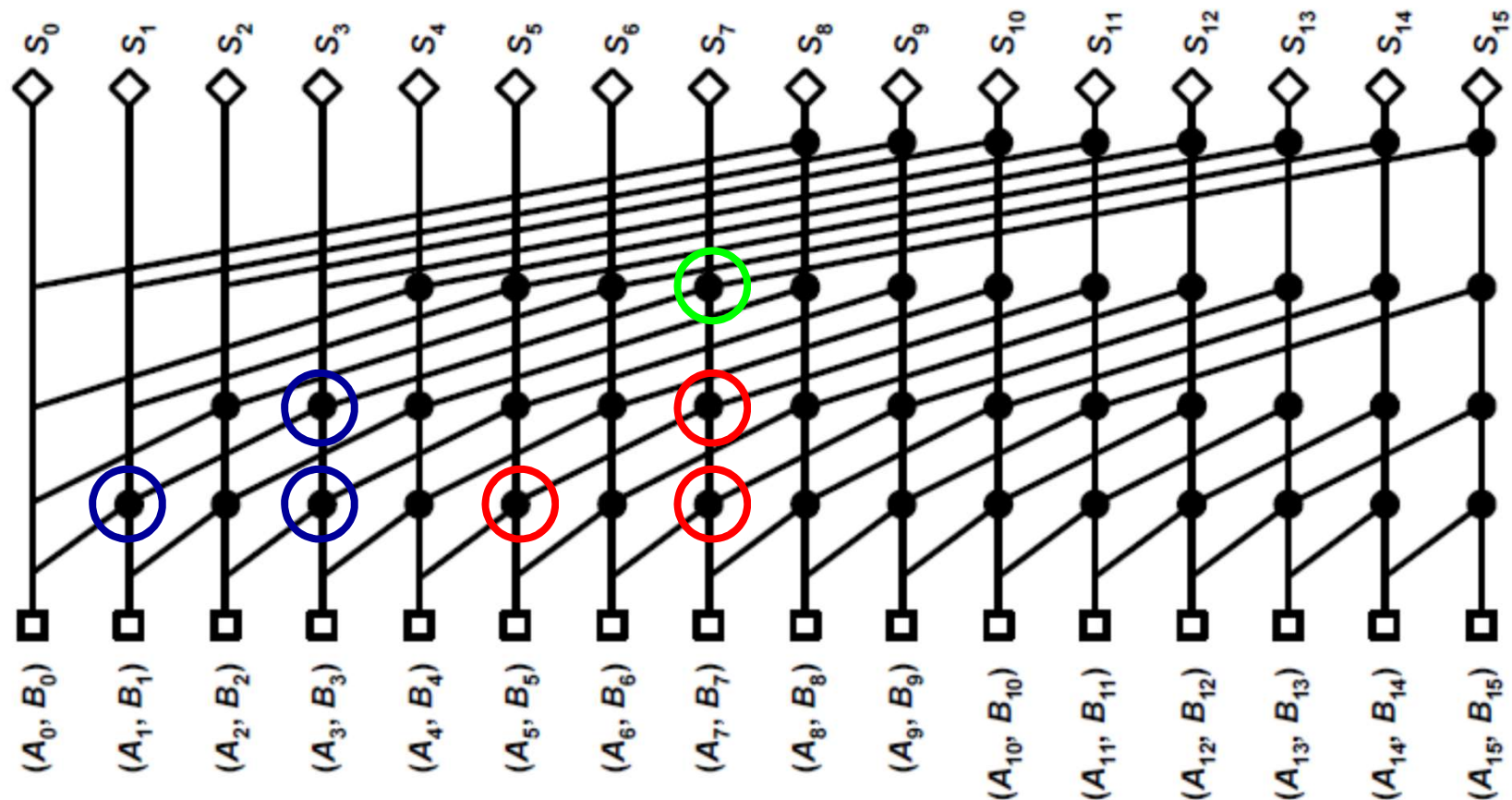
$$\begin{aligned}(C_{o,3},0) &= (G_3, P_3) \bullet (G_2, P_2) \bullet (G_1, P_1) \bullet (G_0, P_0) \bullet (C_{i,0},0) \\ &= (G_{3:2}, P_{3:2}) \bullet (G_{1:0}, P_{1:0}) \bullet (C_{i,0},0)\end{aligned}$$

对数超前进位加法器



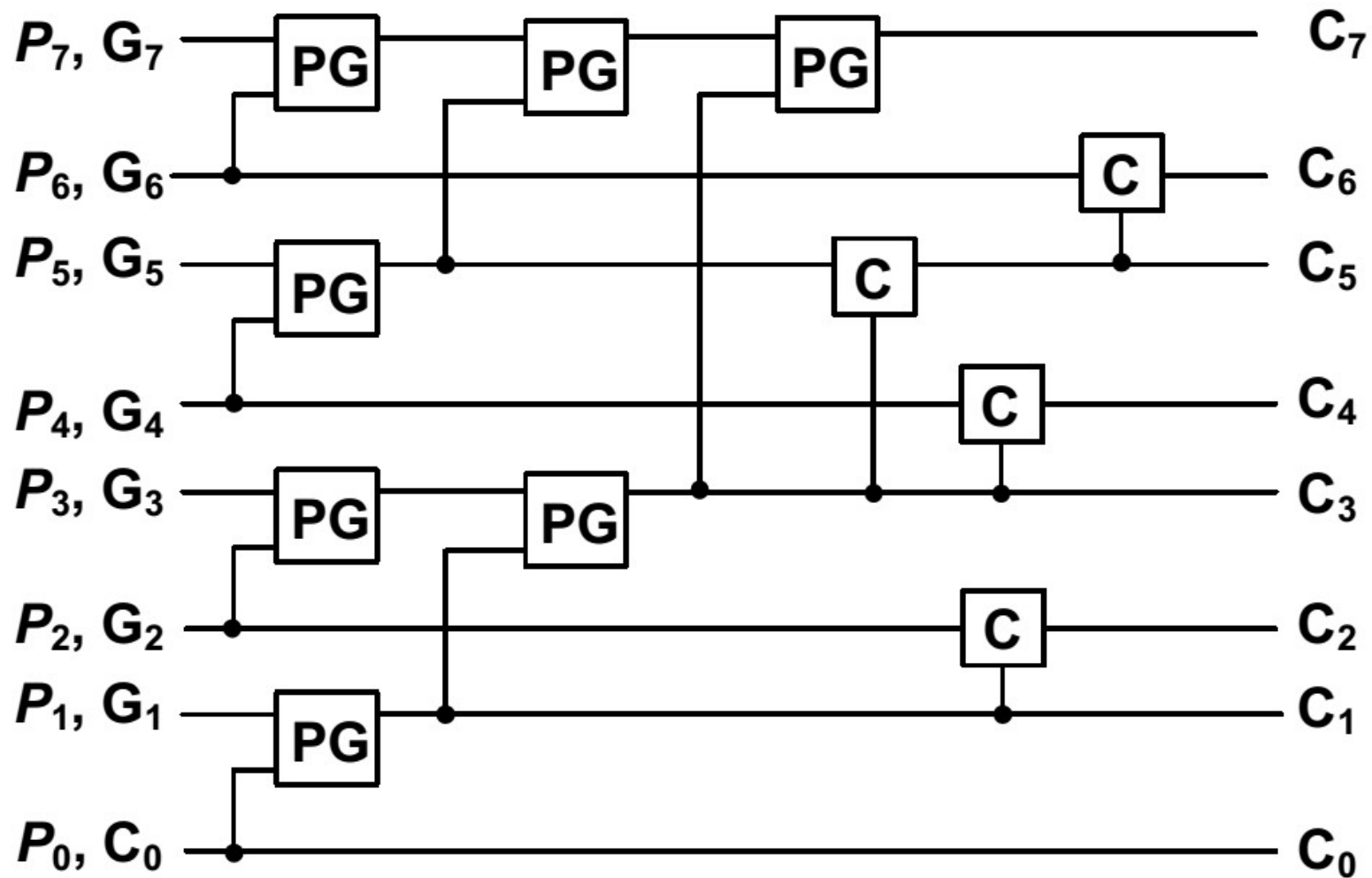
$$t_p \sim \log_2(N)$$

对数超前进位加法器



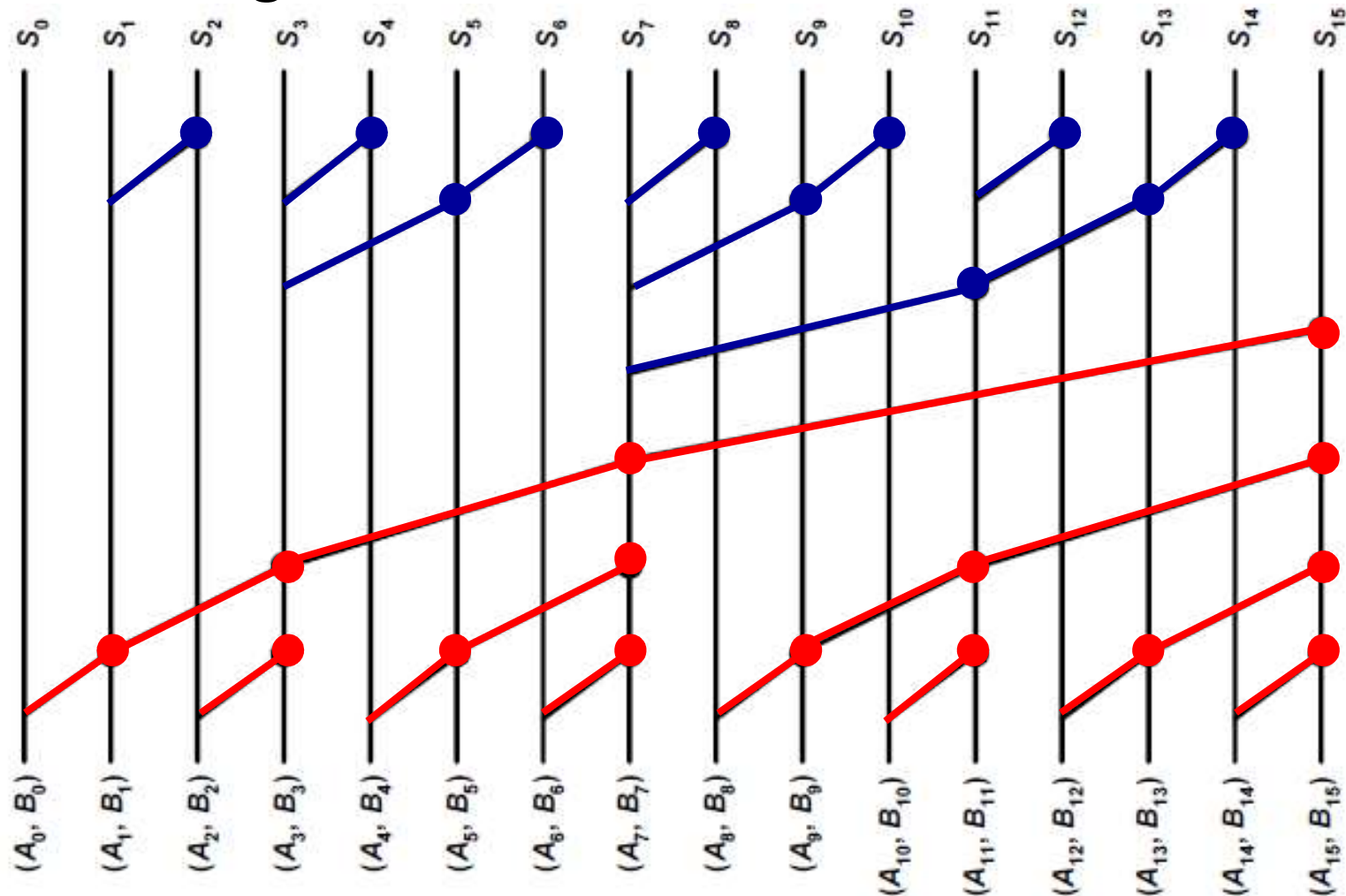
$$\begin{aligned}
 (C_{o,7}, 0) &= (G_{7:6}, P_{7:6}) \bullet (G_{5:4}, P_{5:4}) \bullet (G_{3:2}, P_{3:2}) \bullet (G_{1:0}, P_{1:0}) \bullet (C_{i,0}, 0) \\
 &= (G_{7:4}, P_{7:4}) \bullet (G_{3:0}, P_{3:0}) \bullet (C_{i,0}, 0) = (G_{7:0}, P_{7:0}) \quad \boxed{C_{i,0} = 0}
 \end{aligned}$$

对数超前进位加法器



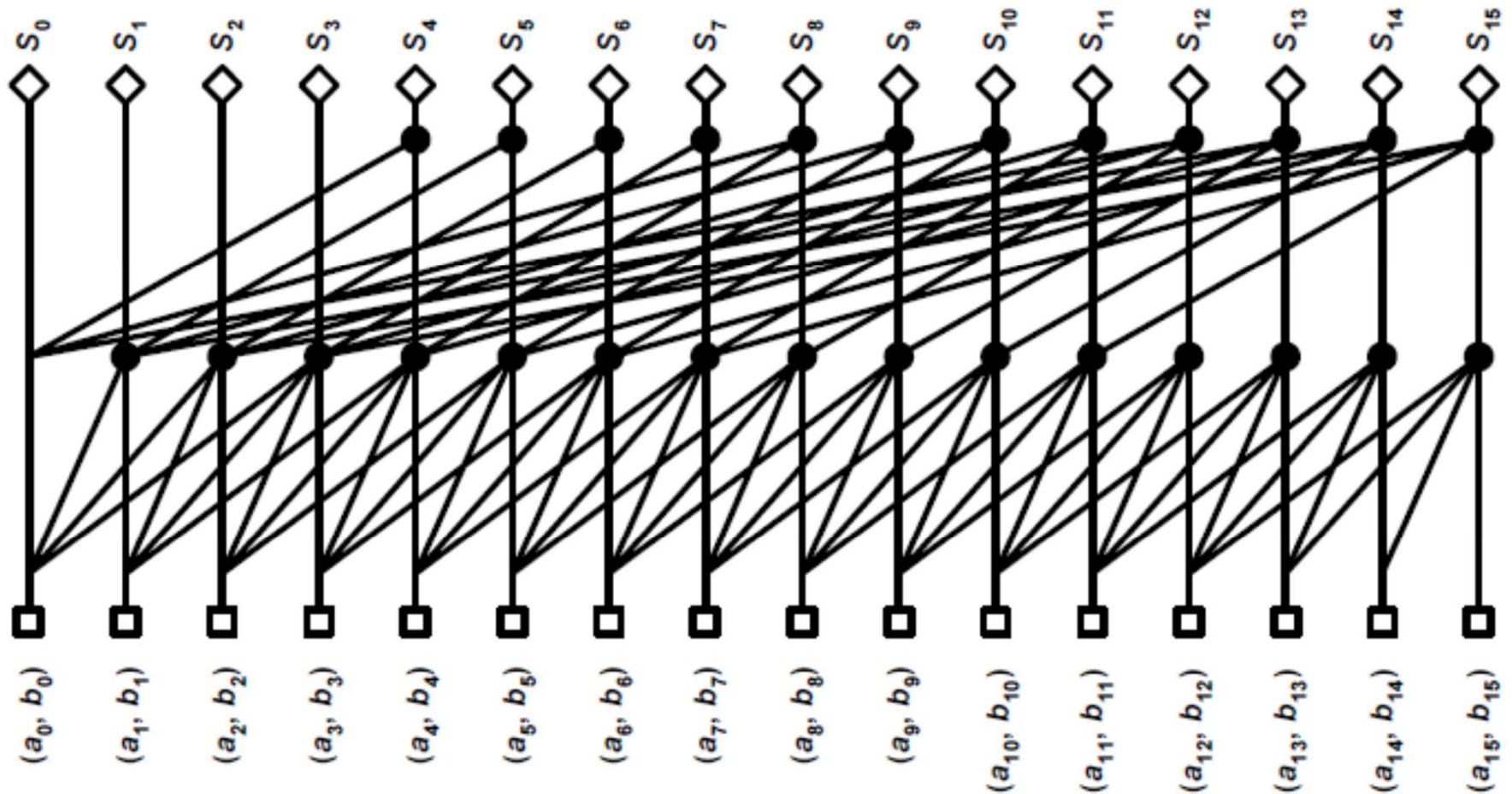
对数超前进位加法器

Brent-Kung树结构



对数超前进位加法器

四进制Kogge-Stone树结构



十进制加法器

■ BCD加法器

- 考虑两个十进制数 **BCD** 码的算术加法，包括来自前一级的进位输入
- 将两个**BCD**码加到一个**4**位二进制加法器上，该加法器就会产生一个二进制和，而且计算结果范围为**0**至**19**

十进制加法器

■ BCD加法器

- 考虑两个十进制数的每一位，并考虑低一级的进位
- 将两个BCD数相加，如果和大于9，就会产生进位，直到19

K	二进制和				BCD 和					十进制数
	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

十进制加法器

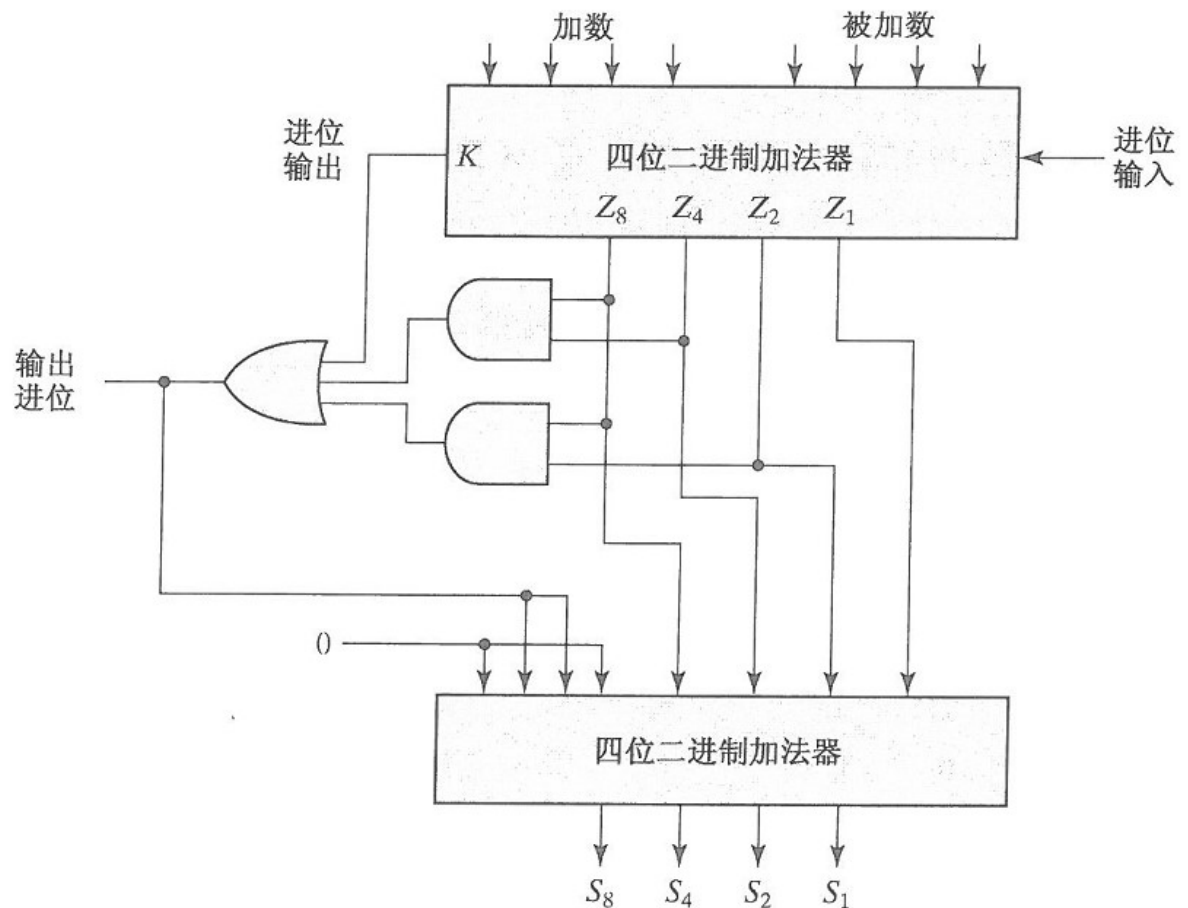
■ BCD加法器

- 要找到一个规则，将**二进制和**转换成正确的**BCD和**
 - ◆ 当**二进制和**等于或者小于**1001**时，其对应的**BCD**数是相等的，不需要转换
 - ◆ 当**二进制和**大于**1001**时，得到的是一个无效的**BCD**表示。需要将二进制数**6(0110)**与二进制和数相加，得到正确的**BCD**表示，并产生所需的输出进位
- 校正条件： $C = K + Z_8Z_4 + Z_8Z_2$ 。当 **$C=1$** 时，需要将**0110**与二进制和相加并给下一级提供一个输出进位

十进制加法器

■ BCD加法器

- 要找到一个
 - ◆ 当二进制相等的,
 - ◆ 当二进制需要将二BCD表示
- 校正条件:
0110与二进



- 一个并行的 n 位十进制加法器需要 n 级 **BCD** 加法器，其中低一级输出进位必须接到高一级的输入进位上

二进制减法器

- 数的补码表示
- 符号二进制数
- 二进制减法器

补码

- 简化减法运算和逻辑运算，从而简化电路，降低复杂度
- 两种类型补码
 - 基数补码，或 r 的补码
 - 基数减1补码（基数反码），或 $(r-1)$ 的补码
- 二进制数：2的补码和1的补码（反码）
- 十进制数：10的补码和9的补码（反码）

补码

■ 基数反码

- 给定一个有 n 个数字的 r 进制数 N ，其 $(r-1)$ 补码定义为 $(r^n-1)-N$
- 十进制数， $r = 10$ ， $(r-1) = 9$
546700的反码为： $999999 - 546700 = 453299$
012398的反码为： $999999 - 012398 = 987601$
- 二进制数， $r = 2$ ， $(r-1) = 1$
1011000的反码为： $1111111 - 1011000 = 0100111$
0101101的反码为： $1111111 - 0101101 = 1010010$
二进制数的反码在形式上就是将1改为0、将0改为1
- 八进制或十六进制数的基数反码就是用7或F减去该数每一位数字得到

补码

■ 基数补码

- 给定一个有 n 个数字的 r 进制数 N ，其 r 补码定义为 $r^n - N$ ， r 的补码实质上就是基数反码加上1
- 十进制数：保留最低有效位上的0不变，用10减去第一个非零有效数字，用9减去所有其它较高位上有效数字

546700的补码为： $1000000 - 546700 = 453300$

012398的补码为： $1000000 - 012398 = 987602$

- 二进制数：不改变所有最低位的0和第一个1，其它更高位上的有效数字用相反的数字来替代，

1011000的补码为： $10000000 - 1011000 = 0101000$

0101101的补码为： $10000000 - 0101101 = 1010011$

二进制数的补码在形式上就是保留所有最低有效位的0和第一个1，其它更高位上所有的1改为0、0改为1

补码

■ 补码减法

- 两个 n 位的 r 进制无符号数减法 $M-N$:

- ◆ 被减数 M 与减数 N 的补码相加, 即

$$M + (r^n - N) = (M - N) + r^n$$

- ◆ 若 $M \geq N$, 结果和将产生进位 r^n , 可以舍弃, 剩下 $M-N$ 的结果

- ◆ 若 $M < N$, 和就不会产生进位, 等于 $r^n - (N - M)$, 即 $(N-M)$ 的补码。取和的补码, 在其前加一负号

- 十进制数补码减法:

- 1) $72532 - 3250$; 2) $3250 - 72532$

- 二进制数补码减法: $X=1010100$, $Y=1000011$

- 1) $X - Y$; 2) $Y - X$

带符号位的二进制数

- 有符号数：二进制数最左边的一比特表示其符号位，**0**表示正号，**1**表示负号
 - 有符号正数有唯一的表示法
 - 例：8比特二进制数表示+9：**0 0001001**
 - 负数有多种不同的表示方法
 - 例：8比特二进制数表示-9
 - 原码表示法（符号-数值）：**1 0001001**
 - 反码表示法（符号-反码）：**1 1110110**
 - 补码表示法（符号-补码）：**1 1110111**
- 计算机系统中通常采用符号-补码表示法

带符号位的二进制数

■ 算术加法

两个以**符号-补码**表示的二进制符号数的加法，就是把两个数相加，包括它们的符号位，符号位上产生的进位要舍弃

$$\begin{array}{r} + 6 \quad 00000110 \\ + 13 \quad 00001101 \\ \hline + 19 \quad \textcircled{0}0010011 \\ \text{符号位} \\ + 6 \quad 00000110 \\ - 13 \quad 11110011 \\ \hline - 7 \quad \textcircled{1}1111001 \\ \text{符号位} \end{array}$$

$$\begin{array}{r} - 6 \quad 11111010 \\ + 13 \quad 00001101 \\ \hline + 7 \quad \textcircled{0}0000111 \\ \text{符号位} \\ - 6 \quad 11111010 \\ - 13 \quad 11110011 \\ \hline - 19 \quad \textcircled{1}1101101 \\ \text{符号位} \end{array}$$

带符号位的二进制数

■ 算术减法

两个以符号-补码表示的二进制符号数的减法非常简单：
取减数的补码（包括符号位），将其与被减数（包括符号位）相加，舍弃符号位上产生的进位

例： $(-6) - (-13) = +7$

$(11111010) - (11110011) = (11111010) + (00001101)$

$$\begin{array}{r} 11111010 \\ + 00001101 \\ \hline 100000111 \end{array}$$

舍弃

二进制加减器

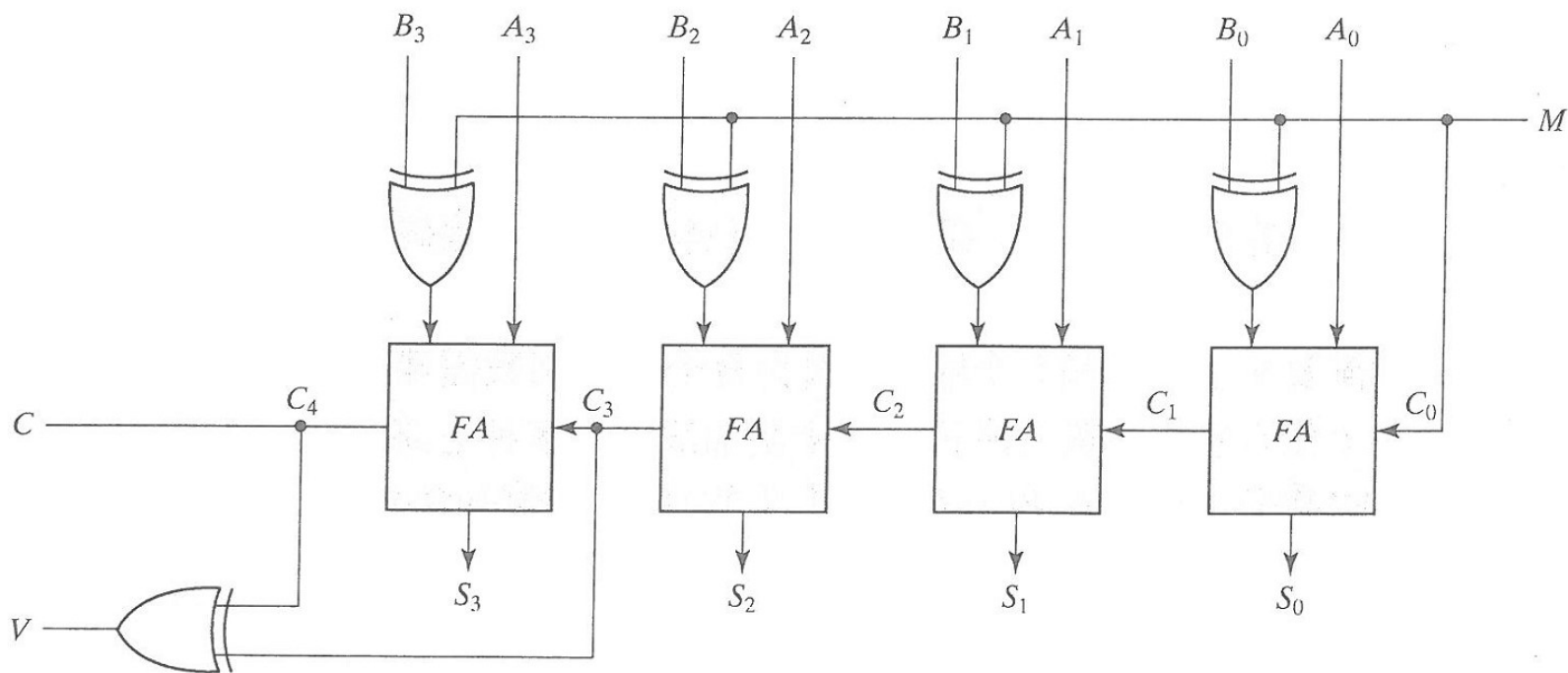
■ 二进制减法器

- 无符号二进制数的减法可通过补码方式很方便地实现。减法 $A-B$ 可通过将 B 的补码与 A 相加来实现。二进制数补码的实现可以先求其反码，然后在反码的最低有效位上加 1 。反码可通过非门实现， 1 可以通过输入进位与和相加
- 减法 $A-B$ 的实现电路包括一个加法器，并且在每个数据输入 B 与其对应的全加器输入之间都有一个非门。当进行减法运算时，输入进位 C_0 必须要等于 1 。这样，该运算就变成了 A 加上 B 的反码，再加 1
- 对于无符号数，如果 $A \geq B$ 就给出 $A-B$ ；若 $A < B$ ，则得到 $B-A$ 的补码。对于带符号数，结果就是 $A-B$

二进制加减器

■ 二进制减法器

- 加法和减法运算可包含在一个普通的二进制加法器电路里。通过将每个全加器加上一个异或门来实现



二进制加减器

■ 溢出

- 如果两个 n 位数相加的和是 $n+1$ 位，则称为溢出
- 要检测两个二进制数相加后是否溢出，取决于这些数是带符号数还是无符号数
 - ◆ 当两个无符号数相加时，溢出要在最高有效位的最后进位检测
 - ◆ 在带符号数的情况下，最左边位总是代表符号，且负数用补码表示。当两个带符号数相加时，符号位被当作数的一部分，且最后进位不表示为溢出
- 若加法运算中有一个数是正数而另一个数为负数，就不会产生溢出。若两个数都为正数或者都为负数，则相加的结果就可能产生溢出

二进制加减器

■ 溢出

● 例：



应该为正数的 8 位数却有一个负的符号(第8 位)，而本该为负数的 8 位数却有了一个正的符号；如果符号位上的进位作为结果的有效位，这样获得的 9 位结果才是正确的

溢出可以由进入到符号位上的进位和符号位上的输出进位来检测。若这两个进位不相等，则产生了溢出